Chapter 14 — Interview Question Bank

How to use this bank

This chapter collects the data-systems questions that come up again and again in senior backend, data-engineering and system-design interviews. They are the questions you see in LeetCode Discuss interview-experience posts, in public system-design and database question lists, and in the "explain X" / "X vs Y" rounds that follow a design exercise.

- **Wording.** Each question is written the way interviewers usually ask it, so you recognise it when you hear it.
- **★ = asked very often.** These showed up across many sources. If you are short on time, drill only the ★ questions first.
- **Answers.** Each answer is short enough to say out loud in 30–60 seconds. Every new term is defined the first time it appears. The answers are written for this book and follow the book's explanations.
- **More.** The *More* link points to the chapter of these notes that explains the topic in full. A few questions (for example the transactional outbox, Kafka's exactly-once settings, Raft's details) go beyond the book; they still get a full answer here, and the link points to the closest chapter or to Chapter 0a — System Design Fundamentals.
- **How to drill.** Cover the answer, read the question, say your answer out loud (or write it in two or three lines), then uncover and compare. Mark the ones you missed and repeat them the next day. For the fast-revision version of the same material, use the Interview Edition.
- **The senior move.** Interviewers at senior level want three things in every answer: the definition, the trade-off ("it costs X to get Y"), and a real system that makes that choice (PostgreSQL, Cassandra, Kafka, ZooKeeper, Spanner…). The answers below try to give all three.

Group	Questions
Reliability, scalability and maintainability	Q1–Q12
Data models and query languages	Q13–Q24
Storage engines and indexes (LSM vs B-tree)	Q25–Q42
OLTP vs OLAP and column storage	Q43–Q50
Encoding and schema evolution	Q51–Q62
Replication	Q63–Q84
Partitioning (sharding)	Q85–Q98
Transactions and isolation anomalies	Q99–Q120
The trouble with distributed systems	Q121–Q134
Consistency and consensus	Q135–Q156
Batch processing	Q157–Q166
Stream processing, CDC and event sourcing	Q167–Q184
Exactly-once, idempotency and data integration	Q185–Q194
Scenario and design questions	Q195–Q206

Reliability, scalability and maintainability

Q1. ★ What do reliability, scalability and maintainability mean for a data system? **Reliability** means users still get the correct result, fast enough, while disks, code or operators are misbehaving.

Scalability means having reasonable ways to cope as load grows — data volume, traffic or complexity. **Maintainability** means many different people can work on the system over years productively: operate it, understand it and change it. A good answer adds that these are not features you bolt on; each is a set of design choices with costs. → *More:* Ch 1 — Reliable, Scalable, and Maintainable Applications

Q2. ⋆ What is the difference between a fault and a failure? A **fault** is one component deviating from its spec — a disk dies, a process crashes, a network link drops packets. A **failure** is when the system as a whole stops giving the required service to the user. Fault-tolerant design aims to stop faults from turning into failures, for example by replicating data so that one dead disk does not lose anything. You cannot reduce faults to zero, so you design to tolerate them, and you may even inject faults on purpose (chaos engineering, Netflix's Chaos Monkey) to test the tolerance. → *More:* Ch 1 — Reliable, Scalable, and Maintainable Applications

Q3. What kinds of faults should a data system expect? Three kinds. **Hardware faults** (disks, RAM, power, network) are mostly random and independent, so redundancy handles them; at scale they are constant — a cluster of 10,000 disks loses about one per day. **Software errors** are systematic: one bug or one bad input can crash every node at once (for example a leap-second bug), so they are more dangerous. **Human errors** (bad config pushes) cause the largest share of outages; you reduce them with good APIs, sandboxes, fast rollback, gradual rollouts and monitoring. → *More:* Ch 1 — Reliable, Scalable, and Maintainable Applications

Q4. ⋆ Why are percentiles better than averages for describing response time? The average (mean) hides how many users had a bad experience. The **median (p50)** tells you what a typical request sees; **p95, p99, p999** (the tail) tell you how slow the slowest 5%, 1% and 0.1% are. Tail latency matters because the slowest requests often belong to the most valuable users (they have the most data), and because of **tail latency amplification**: if one page calls 100 backends in parallel, the page is as slow as the slowest call, so a 1-in-100 slow call hits most pages. Amazon famously set targets at p999 for internal services. → *More:* Ch 1 — Reliable, Scalable, and Maintainable Applications

Q5. What is the difference between latency and response time? Throughput? Response time is what the client sees: network delay + queueing delay + service time. **Latency** strictly is the time a request waits to be handled (it is "latent"), though people often use the two words loosely. **Throughput** is how many requests (or bytes) the system handles per second. A system can have high throughput and bad latency, for example when deep queues keep the CPU busy while each request waits a long time. Queueing is also why you should measure response time on the client side. → *More:* Ch 1 — Reliable, Scalable, and Maintainable Applications

Q6. ⋆ Explain the Twitter home-timeline example. Fan-out on write or fan-out on read? Fan-out on read: when a user opens their timeline, query all the people they follow and merge the tweets — cheap writes, expensive reads. **Fan-out on write**: when someone tweets, push it into a cached timeline for each follower — cheap reads, expensive writes. Because reads were about 100× more common than writes, Twitter moved to fan-out on write, but celebrities with tens of millions of followers made one tweet into millions of writes, so they ended with a **hybrid**: fan-out on write for most users, fan-out on read for celebrities, merged at read time. The lesson: the right design depends on the **load parameters** (here, the distribution of followers per user). → *More:* Ch 1 — Reliable, Scalable, and Maintainable Applications

Q7. ⋆ Vertical vs horizontal scaling — when do you choose each? Vertical scaling (scale up) means a bigger machine: simple, no distribution problems, but it gets expensive fast and has a hard ceiling and a single point of failure. **Horizontal scaling (scale out)** means more machines sharing the load (**shared-nothing**): almost no ceiling and better fault tolerance, but you take on replication, partitioning, and distributed-systems problems. Stateless services scale out easily; stateful databases are harder. A common senior answer is "scale up until it hurts, then partition", since a single big database is simpler to run and many systems never outgrow it. → *More:* Ch 1 — Reliable, Scalable, and Maintainable Applications

Q8. What does "elastic" mean, and when is manual scaling better? An **elastic** system adds or removes resources automatically when load changes. It helps when load is unpredictable. Manual scaling (a human decides to add nodes) is simpler and has fewer operational surprises — for example, automatic rebalancing of a database can trigger a cascade of data movement at the worst moment. Many databases deliberately keep rebalancing manual or human-approved. → *More:* Ch 1 — Reliable, Scalable, and Maintainable Applications

Q9. What are operability, simplicity and evolvability? These are the three parts of maintainability. **Operability**: make it easy for operations teams to keep the system healthy — good monitoring, automation, documentation, predictable behaviour. **Simplicity**: remove **accidental complexity** (complexity that comes from the implementation, not the problem) using good abstractions. **Evolvability**: make it easy to change the system as requirements change, which depends heavily on simplicity and on things like schema evolution and rolling upgrades. → *More:* Ch 1 — Reliable, Scalable, and Maintainable Applications

Q10. How do you calculate availability, and what does "four nines" allow? Availability is the fraction of time the service works: uptime / total time. **99.9%** ("three nines") allows about 8.8 hours of downtime per year (~43 minutes per month); **99.99%** allows about 53 minutes per year (~4.3 minutes per month); **99.999%** about 5.3 minutes per year. Two components in series multiply: two services at 99.9% each give about 99.8%. Redundant components in parallel improve it: two independent 99% replicas give $1 - 0.01^2 = 99.99\%$, if their failures really are independent. → *More:* Ch 0a — System Design Fundamentals

Q11. What are SLI, SLO and SLA? What is an error budget? An **SLI** (service level indicator) is a measured number, such as p99 latency or success rate. An **SLO** (objective) is the internal target for it ("p99 < 200 ms over 30 days"). An **SLA** (agreement) is a contract with customers with penalties if it is missed, usually looser than the SLO. The **error budget** is 100% minus the SLO: the amount of unreliability you are allowed to "spend" on risky launches; when it is used up, you freeze risky changes. → *More:* Ch 0a — System Design Fundamentals

Q12. How would you estimate the storage and QPS for a system? (back-of-the-envelope) Start from users and actions: daily active users × actions per user per day ÷ 86,400 seconds ≈ average QPS (queries per second); multiply by 2–10 for peak. Storage = records per day × bytes per record × retention days × replication factor. Round aggressively (a day ≈ 10^5 seconds) and state assumptions out loud. Example: 100M users × 10 writes/day = 10^9 writes/day ≈ 12,000 writes/s average; at 1 KB each that is 1 TB/day, 365 TB/year before replication. → *More:* Ch 0a — System Design Fundamentals

Data models and query languages

Q13. ⋆ Relational vs document databases — when do you choose each? A **relational** database (PostgreSQL, MySQL) stores rows in tables with a fixed schema and is strong at joins, many-to-many relationships and transactions. A **document** database (MongoDB, Couchbase) stores self-contained JSON-like documents; it fits data that is a tree of one-to-many relationships loaded together (a résumé with its jobs and schools), gives **locality** (one read fetches everything) and a flexible schema. Choose documents when your data has a document shape and few cross-document relationships; choose relational when you have many references between entities. The two have converged: PostgreSQL has JSONB, MongoDB has `$lookup` joins and multi-document transactions. → *More:* Ch 2 — Data Models and Query Languages

Q14. What is the object-relational mismatch (impedance mismatch)? Application code works with objects (nested, with lists inside), while relational tables are flat rows. The translation layer between them is awkward — that is the **impedance mismatch**. ORMs (object-relational mappers such as Hibernate, SQLAlchemy) hide some of it but not all, and can cause problems like the N+1 query pattern. JSON documents reduce the mismatch for tree-shaped data. → *More:* Ch 2 — Data Models and Query Languages

Q15. ⋆ Schema-on-read vs schema-on-write? Schema-on-write (relational): the database enforces the schema when data is written, so all stored data matches it — like static type checking. **Schema-on-read** (document databases): the data has an implicit structure that the code interprets when it reads it — like dynamic type checking. Schema-on-read is handy when items are heterogeneous or the structure is controlled by external systems. Changing structure: with schema-on-write you run a migration (`ALTER TABLE`); with schema-on-read the code handles both old and new shapes. → *More:* Ch 2 — Data Models and Query Languages

Q16. ⋆ What is normalization, and when would you denormalize? Normalization means storing each fact once and referring to it by ID (for example storing a `region_id` instead of the text "Greater Seattle Area" in every profile). It avoids duplicated data getting out of sync and makes updates cheap. **Denormalization** copies data into the places where it is read, so reads need no join — faster reads at the cost of more complex, slower writes and the risk of inconsistency. Denormalize for read-heavy paths

(feeds, search indexes, caches) and treat the copies as derived data you can rebuild. → *More:* Ch 2 — Data Models and Query Languages

Q17. How do one-to-many, many-to-one and many-to-many relationships affect the data-model choice? One-to-many (a user has many positions) fits a nested document naturally. Many-to-one (many people live in one region) and many-to-many (people and organisations) need references and joins. Document databases historically had weak join support, so the application had to do the joins itself. As an application grows, data tends to become more interconnected, which pushes toward relational or graph models. → *More:* Ch 2 — Data Models and Query Languages

Q18. Declarative vs imperative query languages — why does SQL being declarative matter? A **declarative** language (SQL, CSS) says *what* result you want, not *how* to compute it. An **imperative** language spells out each step and order. Declarative queries let the database's **query optimizer** choose indexes and join methods, change them later without changing your query, and run the query in parallel across cores or machines. Imperative code fixes the algorithm and the order, which is hard to parallelise. → *More:* Ch 2 — Data Models and Query Languages

Q19. ⋆ When would you use a graph database? When many-to-many relationships are the main point of the data and queries follow paths of unknown length: social networks ("friends of friends"), fraud rings, recommendation, road networks, knowledge graphs. In the **property graph** model (Neo4j) each vertex and edge has a label and properties; **Cypher** is its declarative query language. In SQL you can do variable-length traversals with recursive common table expressions (`WITH RECURSIVE`), but the query becomes much longer. → *More:* Ch 2 — Data Models and Query Languages

Q20. Property graph vs triple-store? A **property graph** stores vertices and edges, each with a set of key-value properties. A **triple-store** stores everything as (subject, predicate, object) triples, for example (`lucy`, `marriedTo`, `alain`) or (`lucy`, `age`, `33`); it is queried with **SPARQL** and is the basis of RDF and the semantic web. They are largely equivalent ways to model the same graph. **Datalog** is an older, rule-based query language that underlies systems like Datomic. → *More:* Ch 2 — Data Models and Query Languages

Q21. Write a recursive SQL query to find all of a node's descendants.

```
WITH RECURSIVE sub(id) AS (
  SELECT id FROM category WHERE id = 42          -- start node
  UNION
  SELECT c.id FROM category c JOIN sub s ON c.parent_id = s.id
)
SELECT id FROM sub;
```

The non-recursive part seeds the result; the recursive part joins children of rows already found until no new rows appear. `UNION` (not `UNION ALL`) also stops cycles from looping forever. → *More:* Ch 2 — Data Models and Query Languages

Q22. What was MapReduce querying in MongoDB, and why did aggregation pipelines replace it? MapReduce querying let you pass two JavaScript functions (map and reduce) to run over matching documents. It was neither fully declarative nor fully imperative, and writing two coordinated functions is harder than one query, and the database can optimise it less. MongoDB added the declarative **aggregation pipeline** (`$match`, `$group`, `$sort` …), and map-reduce was deprecated in MongoDB 5.0. → *More:* Ch 2 — Data Models and Query Languages

Q23. What does "NoSQL" really mean, and what is polyglot persistence? "NoSQL" started as a Twitter hashtag for a 2009 meetup and is now read as "Not Only SQL". It groups databases built for greater scalability, open-source, specialised queries or schema flexibility. **Polyglot persistence** means using several data stores side by side in one application, each for what it is good at (PostgreSQL for orders, Redis for cache, Elasticsearch for search). → *More:* Ch 2 — Data Models and Query Languages

Q24. How do you store and query JSON in PostgreSQL efficiently? Use the `jsonb` type (binary, parsed once, supports indexing) rather than `json` (stored as text). Query with operators like `->>` and containment `@>`, and add a **GIN index** (generalised inverted index) for containment queries. Keep fields you filter or join on often as real columns; use `jsonb` for the truly variable parts.

```
CREATE INDEX ON events USING gin (payload jsonb_path_ops);
SELECT * FROM events WHERE payload @> '{"type": "signup"}';
```

→ *More:* Ch 2 — Data Models and Query Languages

Storage engines and indexes (LSM vs B-tree)

Q25. ⋆ How does a database store and find data on disk? Explain the simplest possible database. The simplest database appends `key,value` lines to a file (a **log**: an append-only sequence of records) and reads by scanning for the last line with that key. Appending is very fast, but reading is O(n). An **index** is an extra structure derived from the data that speeds up reads. Every index slows writes, because it must be updated on each write — this read/write trade-off is the core of storage-engine design. → *More:* Ch 3 — Storage and Retrieval

Q26. What is a hash index (Bitcask)? Keep an in-memory hash map from each key to the byte offset of its latest value in the append-only data file. A read is one hash lookup plus one disk seek; writes are appends. The log is split into **segments**; old segments are **compacted** (duplicate keys thrown away, keeping only the latest) and merged in the background. Deletes write a **tombstone** (a special deletion marker). Limits: all keys must fit in RAM, and range queries are inefficient. Bitcask (Riak's default engine) works this way. → *More:* Ch 3 — Storage and Retrieval

Q27. ⋆ What is an LSM-tree and how does it work? An **LSM-tree** (log-structured merge-tree) buffers writes in an in-memory sorted structure called the **memtable** (for example a red-black tree or skip list). When it gets big (a few MB), it is written to disk as an **SSTable** (sorted string table: a file of key-value pairs sorted by key). Reads check the memtable, then the newest SSTable, then older ones. Background **compaction** merges SSTables, discarding overwritten and deleted values. A **write-ahead log** protects the memtable from crashes. Used by LevelDB, RocksDB, Cassandra, HBase, ScyllaDB. → *More:* Ch 3 — Storage and Retrieval

Q28. ⋆ How does a B-tree work? A **B-tree** splits the database into fixed-size **pages** (traditionally 4 KB; PostgreSQL uses 8 KB, InnoDB 16 KB). Each page holds keys and references to child pages covering key ranges; leaves hold the values (or pointers to them). A lookup walks from the root to a leaf — with a branching factor of a few hundred, a 4-level tree of 4 KB pages can hold up to 256 TB. Updates overwrite the page in place; if a page is full it **splits** into two and the parent is updated. A **write-ahead log (WAL)** makes this crash-safe. B-trees are the standard index in almost every relational database. → *More:* Ch 3 — Storage and Retrieval

Q29. ⋆ LSM-tree vs B-tree — compare them. Which would you pick for a write-heavy workload?

	LSM-tree	B-tree
Writes	Sequential appends, high throughput	Random in-place page writes
Reads	May check several SSTables (Bloom filters help)	One root-to-leaf path, predictable
Write amplification	Compaction rewrites data several times	Each write = WAL + whole page
Space	Compresses well, less fragmentation	Pages partly empty after splits
Latency risk	Compaction can stall or fall behind	Steadier
Transactions	Key may exist in several places	Each key in one place, easy range locks

Rule of thumb: LSM for write-heavy, append-like ingestion (metrics, logs, time series, messaging); B-tree for read-heavy OLTP with predictable latency. Always benchmark with your real workload. → *More:* Ch 3 — Storage and Retrieval

Q30. ⋆ What is write amplification, and why does it matter? Write amplification is the number of bytes actually written to disk per byte the application writes. B-trees write the WAL and then a whole page (even for a tiny change); LSM-trees rewrite data during each compaction level. On SSDs, which wear out after a limited number of write cycles, and on disks with limited write bandwidth, high write amplification directly reduces throughput and lifetime. Read amplification (disk reads per query) and space amplification are the other two sides of the same trade-off. → *More:* Ch 3 — Storage and Retrieval

Q31. ⋆ What is a Bloom filter, and why do LSM-trees use one? A **Bloom filter** is a small bit array plus several hash functions that answers "is this key in the set?" with either "definitely not" or "probably yes" — no false negatives, some false positives. An LSM-tree keeps one per SSTable, so a read for a

missing key can skip almost every file without touching disk. With about 10 bits per key the false-positive rate is about 1%. You cannot delete from a plain Bloom filter (a counting Bloom filter can). → *More:* Ch 3 — Storage and Retrieval

Q32. Size-tiered vs leveled compaction? Size-tiered compaction merges newer, smaller SSTables into older, larger ones of similar size: good write throughput, but more space used temporarily and more files to check on reads (Cassandra's default STCS). **Leveled** compaction keeps levels (L0, L1, L2…), each about 10× bigger than the last, with non-overlapping key ranges within a level: lower read and space amplification, more write amplification (LevelDB, RocksDB default). Pick leveled for read-heavy or update-heavy data, size-tiered for write-heavy insert-mostly data. → *More:* Ch 3 — Storage and Retrieval

Q33. How does a B-tree survive a crash in the middle of a page split? A page split writes several pages; a crash midway could leave an orphan page or a broken tree. The B-tree first records every change in a **write-ahead log** (WAL, also called a redo log), an append-only file that is flushed to disk before the pages change. After a crash, the database replays the WAL to restore a consistent tree. Some engines use **copy-on-write** instead (write modified pages to new locations, then atomically switch the parent pointer), as LMDB does; this also helps snapshot reads. → *More:* Ch 3 — Storage and Retrieval

Q34. ⋆ What are clustered, non-clustered and covering indexes? A **clustered index** stores the full row inside the index's leaf (InnoDB's primary key is clustered; there is one per table). A **non-clustered (secondary) index** stores a reference — in InnoDB the primary key, in PostgreSQL a pointer into the **heap file** (the unordered place rows live). A **covering index** (index with included columns) stores enough extra columns to answer a query from the index alone ("index-only scan"). Trade-off: faster reads, more storage and slower writes. → *More:* Ch 3 — Storage and Retrieval

Q35. How does a composite (multi-column) index work, and why does column order matter? A **concatenated index** on `(last_name, first_name)` sorts by the first column, then the second within ties — like a phone book. It can answer queries on `last_name`, or on `last_name AND first_name`, but not on `first_name` alone (the **leftmost-prefix rule**). Put equality-filtered columns first, then the range or sort column. For true two-dimensional ranges (latitude and longitude), use a spatial index such as an R-tree, not a composite B-tree. → *More:* Ch 3 — Storage and Retrieval

Q36. Why might adding an index make the system slower? Every index must be updated on each insert, update and delete, which adds write I/O, WAL volume, lock contention and storage. Unused or redundant indexes also take memory away from the buffer cache. And the optimizer may choose a poor index plan. So index only for real query patterns and remove unused ones (PostgreSQL's `pg_stat_user_indexes` shows usage). → *More:* Ch 3 — Storage and Retrieval

Q37. How do full-text search and fuzzy indexes work? Full-text search uses an **inverted index**: a map from each term to the list of documents (postings) containing it. Lucene (behind Elasticsearch and Solr) stores the term dictionary in SSTable-like segments that are merged like an LSM-tree, and keeps the term dictionary in a finite-state automaton similar to a trie, which allows searching for words within a given **edit distance** (fuzzy search). Text is first analysed: split into tokens, lower-cased, stemmed, synonyms added. → *More:* Ch 3 — Storage and Retrieval

Q38. Are in-memory databases fast because they avoid disk reads? Mostly not. A disk-based database with enough RAM already serves reads from the OS page cache. In-memory databases (Redis, VoltDB, MemSQL/SingleStore) are faster because they avoid the cost of **encoding in-memory structures into a disk format**, and they can offer data structures (sets, priority queues) that are hard to build on disk. They stay durable through snapshots, logs on disk or replication. → *More:* Ch 3 — Storage and Retrieval

Q39. What is a write-ahead log and why does almost every database have one? A **WAL** is an append-only file where every change is recorded and flushed (`fsync`) to disk *before* the change is applied to the main data files. If the machine crashes, replaying the WAL restores all committed changes — this gives the "D" (durability) in ACID. Sequential appends are fast, so the database can delay writing the real data pages. The same log is often reused for replication (PostgreSQL streaming replication ships WAL) and for change data capture. → *More:* Ch 3 — Storage and Retrieval

Q40. What does `fsync` do, and why do databases care about it? Writing to a file normally only copies data into the OS page cache; a power cut can lose it. `fsync` forces the data to stable storage and waits until the device says it is written. Databases call it on commit (or in a group commit for many transactions at once) to guarantee durability. Some settings trade safety for speed — for example `synchronous_commit = off` in PostgreSQL or `innodb_flush_log_at_trx_commit = 2` in MySQL can lose the most

recent commits (roughly the last second) on a crash — for the MySQL setting, an OS crash or power loss, not just a mysqld crash. → *More:* Ch 3 — Storage and Retrieval

Q41. How are deletes handled in an LSM-tree, and what problems do tombstones cause? A delete writes a **tombstone** (a marker that the key is deleted) because older SSTables cannot be changed. The value disappears only when compaction merges the tombstone with all older copies. Problems: reads that scan many tombstones get slow (Cassandra warns about this), and in a replicated system a tombstone must not be purged before every replica has seen it, or the deleted data can come back ("zombie" data; Cassandra's `gc_grace_seconds` sets this window). → *More:* Ch 3 — Storage and Retrieval

Q42. How would you design a key-value store's storage engine? (Bitcask / LSM design question) Start with an append-only log plus an in-memory hash index (Bitcask) and say its limits: keys must fit in RAM, no range scans. Then move to an LSM-tree: WAL, memtable, flush to SSTables with a sparse index and a Bloom filter each, background compaction, tombstones for deletes, checksums to detect torn writes, and crash recovery by replaying the WAL. Discuss compaction strategy, how compaction can fall behind under sustained writes, and how you would add replication and partitioning on top. → *More:* Ch 3 — Storage and Retrieval

OLTP vs OLAP and column storage

Q43. ⋆ What is the difference between OLTP and OLAP? OLTP (online transaction processing) serves the application: many small reads and writes, each touching a few rows found by key, low latency, current data. **OLAP** (online analytic processing) serves analysts: few but huge queries that scan millions of rows and aggregate a few columns, over historical data. They need different storage layouts, so companies run OLAP in a separate **data warehouse** loaded by **ETL** (extract-transform-load) from the OLTP databases. → *More:* Ch 3 — Storage and Retrieval

Q44. ⋆ Why is column-oriented storage faster for analytics? A **column store** keeps all values of each column together, instead of each row together. An analytic query usually reads 4 or 5 columns of a table that has hundreds, so it reads only those columns from disk. Columns also compress very well (similar values next to each other), and the CPU can process compressed column chunks in tight loops that fit in cache and use SIMD (**vectorized processing**). Examples: Parquet files, ClickHouse, Snowflake, BigQuery, Redshift, DuckDB. → *More:* Ch 3 — Storage and Retrieval

Q45. What compression techniques do column stores use? Bitmap encoding: for a column with few distinct values, store one bitmap per value (bit i = 1 if row i has that value); `WHERE x IN (a, b)` becomes a bitwise OR. Sparse bitmaps are then **run-length encoded** (store "5 zeros, 2 ones, …"). Other common schemes: dictionary encoding, delta encoding for sorted numbers, and general-purpose compression (LZ4, Zstd). Sorting the data first makes all of them work better. → *More:* Ch 3 — Storage and Retrieval

Q46. What are star and snowflake schemas? In a **star schema** a central **fact table** stores one row per event (a sale, a click) with measures (price, quantity) and foreign keys to **dimension tables** (who, what, where, when — product, store, customer, date). Drawn out, the dimensions surround the fact table like star points. A **snowflake schema** further normalises dimensions into sub-dimensions (product → brand → category). Star schemas are simpler for analysts; fact tables can have trillions of rows and are often 100+ columns wide. → *More:* Ch 3 — Storage and Retrieval

Q47. How does sort order help a column store, and why keep several sort orders? Rows are stored in a chosen order (for example by date, then product), applied to all columns together so the nth entry of each column still belongs to the same row. Sorting lets queries skip whole ranges and makes the first sort column compress extremely well with run-length encoding. Vertica's idea: since data is replicated anyway, store each replica in a different sort order and let each query use the best one. → *More:* Ch 3 — Storage and Retrieval

Q48. How do column stores handle writes? Inserting a row in the middle of sorted, compressed columns would mean rewriting every column file. So writes go first to an in-memory store (row-oriented or sorted, like an LSM memtable); when enough accumulate they are merged with the on-disk column files and written as new files in bulk. Queries read both and combine them. This is the LSM idea applied to columns. → *More:* Ch 3 — Storage and Retrieval

Q49. What are materialized views and data cubes? A **materialized view** stores the actual result of a query on disk (unlike a virtual view, which is just a saved query) and must be updated when the

underlying data changes — that makes writes costlier, so they suit read-heavy warehouses. A **data cube (OLAP cube)** is a special materialized view: a grid of aggregates (for example SUM of sales) grouped by several dimensions at once, so "sales per store per day" is a lookup. It is fast but inflexible: you cannot ask about a dimension that is not in the cube. → *More:* Ch 3 — Storage and Retrieval

Q50. Data warehouse vs data lake vs lakehouse? A **data warehouse** stores cleaned, structured data in a schema designed for analytics (Snowflake, BigQuery, Redshift). A **data lake** stores raw files of any format cheaply in object storage (S3) and applies schema when reading. A **lakehouse** adds warehouse features on top of lake files using an open **table format** — Apache Iceberg, Delta Lake or Apache Hudi — which adds ACID commits, schema evolution and time travel to Parquet files. These table formats are newer than the book, but they build on its columnar-storage ideas. → *More:* Ch 3 — Storage and Retrieval

Encoding and schema evolution

Q51. ⋆ What are backward and forward compatibility? Backward compatibility: newer code can read data written by older code. **Forward compatibility**: older code can read data written by newer code. You need both because during a **rolling upgrade** (updating nodes one at a time) and with mobile clients that never update, old and new versions run at the same time and read each other's data. Backward compatibility is usually easy (you know the old format); forward compatibility is harder (old code must ignore fields it does not know). → *More:* Ch 4 — Encoding and Evolution

Q52. ⋆ JSON vs Protocol Buffers vs Avro — which would you choose and why? JSON is human-readable and universal, but verbose, has no schema by default, cannot tell integers from floats (numbers above 2^{53} lose precision in JavaScript) and has no binary strings. **Protocol Buffers** (and Thrift) use a schema with numbered **field tags**; the encoded bytes contain tags, not names, so they are compact and support evolution via tags. **Avro** has no tags: the writer's and reader's schemas are resolved side by side by field name, which suits dynamically generated schemas (for example, dumping a database table). Choose JSON for public APIs and debugging, Protobuf for service-to-service RPC (gRPC), Avro for data pipelines and Kafka with a schema registry. → *More:* Ch 4 — Encoding and Evolution

Q53. What are the rules for evolving a Protocol Buffers schema safely? Never change or reuse a field's **tag number** — it is the field's identity on the wire. You may add new fields with new tags; old code skips unknown tags (forward compatible), and new code sees a default for missing fields (backward compatible). Delete a field only if it is not required, and mark its number and name `reserved` so nobody reuses them. Changing types is risky: widening int32 to int64 works for new readers, but old readers truncate the value. In proto3 every field is optional; `required` (proto2) is a trap because you can never remove it. → *More:* Ch 4 — Encoding and Evolution

Q54. ⋆ How does Avro handle schema evolution? What is a writer's and reader's schema? Data is encoded with the **writer's schema**; the reading application decodes with its own **reader's schema**. The Avro library compares the two by field name: fields only in the writer's schema are ignored; fields only in the reader's schema are filled with the default value. So you may only add or remove fields that have a **default** value. The reader must know the writer's schema: store it once at the start of a large file, store a version number per database record, or (in Kafka) put a schema ID in every message that points to a **schema registry**. → *More:* Ch 4 — Encoding and Evolution

Q55. What is a schema registry and what compatibility modes does it enforce? A **schema registry** (Confluent Schema Registry, AWS Glue Schema Registry) stores every version of every schema and gives it an ID. Producers write the ID in each message; consumers fetch the schema by ID. Before registering a new version, it checks compatibility: **BACKWARD** (new readers can read old data — the Confluent default), **FORWARD** (old readers can read new data), **FULL** (both), and *_TRANSITIVE variants that check against all past versions, not just the latest. → *More:* Ch 4 — Encoding and Evolution

Q56. Why should you avoid language-specific serialization (Java Serializable, Python pickle)? They tie the data to one language, evolve poorly between versions, are often slow and bulky, and — most importantly — decoding can instantiate arbitrary classes, so reading untrusted bytes can lead to **remote code execution**. Use them only for very short-lived, same-process purposes, if at all. → *More:* Ch 4 — Encoding and Evolution

Q57. What are the three main modes of dataflow between processes?

1. **Through databases**: the writer encodes, a reader (maybe a future version of the code) decodes later — the book calls this sending a message to your future self. 2. **Through services**: REST or RPC calls — the client encodes a request, the server decodes it, then the reverse for the response. 3. **Through asynchronous message passing**: via a message broker (RabbitMQ, Kafka); the sender does not wait. Each mode has its own compatibility and failure concerns. → *More:* Ch 4 — Encoding and Evolution

Q58. What is the danger when old code reads and rewrites a record containing a new field? If an old version of the app reads a record that has a field added by a newer version, decodes it into an object that has no place for that field, and writes it back, the new field is silently lost. The fix: the old code (or its library) must preserve unknown fields when it rewrites the record. Protobuf preserves unknown fields when it re-serialises a message; hand-written ORM code often does not. → *More:* Ch 4 — Encoding and Evolution

Q59. What does "data outlives code" mean for database migrations? Code is redeployed in minutes, but the data in a database may have been written five years ago by a version long gone. Rewriting all data on each schema change is expensive, so most databases allow cheap changes like adding a nullable column with no default (old rows read as null). In practice you do **expand/contract** migrations: add the new column, deploy code that writes both, backfill, switch reads, then drop the old column — each step compatible with the version before it. → *More:* Ch 4 — Encoding and Evolution

Q60. REST vs RPC — what are the problems with making a remote call look like a local function call? RPC (remote procedure call) tries to make a network request look like a local call (location transparency), but a network call is fundamentally different: it can time out with no idea whether the action happened; retries can repeat the action unless it is **idempotent** (safe to apply more than once); latency varies wildly; parameters must be encoded; and client and server may be in different languages. **REST** embraces the network: resources, HTTP methods, caching and status codes. Modern RPC frameworks (gRPC) are explicit about the network — futures/promises, streams, deadlines. → *More:* Ch 4 — Encoding and Evolution

Q61. How do you version an API so old and new clients keep working? For services you usually only need backward compatibility on requests and forward compatibility on responses, because servers are upgraded before clients. Common approaches: version in the URL (`/v2/orders`) or in an `Accept` header; additive changes only within a version (new optional fields, never removed or retyped fields); and keep old versions alive for as long as clients use them (forever for public APIs with mobile clients). With gRPC/Protobuf, follow the tag rules in Q53. → *More:* Ch 4 — Encoding and Evolution

Q62. What is the actor model, and how does it relate to message passing? An **actor** is a unit of logic with private state that communicates only by sending asynchronous messages to other actors, processing one message at a time — so no locks are needed. Distributed actor frameworks (Akka, Orleans, Erlang/OTP) spread actors across nodes, so the same message encoding and the same forward/backward compatibility rules apply during rolling upgrades. → *More:* Ch 4 — Encoding and Evolution

Replication

Q63. ⋆ Why replicate data? Three reasons: **high availability** (keep working when a node or datacentre dies), **lower latency** (keep a copy geographically close to users), and **read scalability** (serve more reads from more copies). Replication is easy when data never changes; all the difficulty is in handling changes. The three main approaches: single-leader, multi-leader and leaderless. → *More:* Ch 5 — Replication

Q64. ⋆ How does leader-follower (single-leader) replication work? One replica is the **leader** (primary/master); all writes go to it. It writes the change locally and sends it to the **followers** (replicas/secondaries) through a **replication log**; each follower applies the changes in the same order. Reads can go to the leader or followers. Used by PostgreSQL, MySQL, MongoDB replica sets, SQL Server availability groups, Kafka partitions and RabbitMQ quorum queues. → *More:* Ch 5 — Replication

Q65. ⋆ Synchronous vs asynchronous replication — trade-offs? With **synchronous** replication the leader waits for the follower to confirm before telling the client the write succeeded: the follower is guaranteed up to date, but one slow or dead follower blocks all writes. With **asynchronous** replication the leader does not wait: fast and available, but if the leader dies, recently confirmed writes not yet

copied are lost. The usual compromise is **semi-synchronous**: one follower synchronous, the rest asynchronous, so at least two nodes always have the latest data. → *More:* Ch 5 — Replication

Q66. How do you add a new follower without downtime? Take a consistent **snapshot** of the leader (ideally without locking — most databases support this for backups), copy it to the new node, then have the follower request all changes since the snapshot's exact position in the replication log (PostgreSQL's log sequence number, MySQL's binlog coordinates or GTID). When it has caught up, it streams changes like any other follower. → *More:* Ch 5 — Replication

Q67. ⋆ What happens when the leader fails? What can go wrong during failover? Failover: detect the failure (usually a timeout), choose a new leader (the most up-to-date follower, by election or by a controller), and reconfigure clients and the old leader to use it. Things that go wrong: with asynchronous replication, the new leader may be missing writes, which are usually discarded — dangerous if other systems already saw them (GitHub's incident where reused auto-increment IDs leaked data into Redis); **split brain**, where two nodes both act as leader; and choosing the timeout — too short causes needless failovers under load, too long means long outages. → *More:* Ch 5 — Replication

Q68. ⋆ What is split brain and how do you prevent it? Split brain is when two nodes both believe they are the leader and both accept writes, so data diverges or is corrupted. Prevent it by requiring a **majority quorum** to elect a leader (a minority side cannot elect one), by **fencing** the old leader (STONITH — "shoot the other node in the head" — cutting its power or access), and by **fencing tokens**: every new leader gets a higher epoch number and storage rejects writes carrying an older one. → *More:* Ch 5 — Replication

Q69. Compare statement-based, WAL shipping, logical (row-based) and trigger-based replication. Statement-based sends the SQL statements; it breaks on nondeterminism (`NOW()`, `RAND()`, auto-increments, triggers). **WAL shipping** sends the storage engine's low-level log; exact, but tied to the storage format, so leader and followers usually must run the same version (no zero-downtime upgrades). **Logical (row-based) log** sends row-level changes ("row X: column Y changed to Z"), separate from the storage format — it allows mixed versions and is the basis of change data capture (MySQL binlog in row mode, PostgreSQL logical replication). **Trigger-based** uses database triggers to copy changes to another table, read by an external process — flexible but slower and more fragile. → *More:* Ch 5 — Replication

Q70. ⋆ What is replication lag, and what problems does it cause? Replication lag is the delay between a write on the leader and its appearance on a follower — usually a fraction of a second, but seconds or minutes under load or failures. Reads from a lagging follower give stale data: the system is only **eventually consistent** (replicas converge if writes stop, with no bound on how long). Three classic anomalies follow: not reading your own write, time going backwards (monotonic reads), and seeing an answer before its question (consistent prefix). → *More:* Ch 5 — Replication

Q71. ⋆ What is read-your-writes consistency and how do you implement it? Read-after-write (read-your-writes) consistency guarantees a user always sees the writes they made themselves (it says nothing about other users). Implementations: read things the user may have modified (their own profile) from the leader; for a minute after a user's last write, read from the leader; or remember the timestamp/log position of the user's last write and only read from a replica that has caught up to it. Cross-device versions are harder: the "last write" metadata must be centralised, and devices may be routed to different datacentres. → *More:* Ch 5 — Replication

Q72. What are monotonic reads? Monotonic reads guarantee a user never sees time go backwards: after reading newer data, they will not later read older data. The anomaly happens when one refresh hits an up-to-date replica and the next hits a lagging one (a comment appears, then disappears). Fix: route each user to the same replica, for example by hashing the user ID (and reroute only if that replica fails). → *More:* Ch 5 — Replication

Q73. What is consistent prefix reads? Consistent prefix reads guarantee that if writes happen in a certain order, anyone reading them sees them in that order. It breaks in partitioned databases, where different partitions replicate at different speeds — an observer might see an answer before the question it answers. Fix: write causally related data to the same partition, or track causal dependencies explicitly. → *More:* Ch 5 — Replication

Q74. ⋆ When would you use multi-leader replication? What are its drawbacks? Use **multi-leader** (master-master) replication for **multi-datacentre** deployments (each region has a leader, writes are local, regions sync asynchronously), for **offline clients** (each device is effectively a leader, like a calendar

app), and for **collaborative editing** (Google Docs-style). Gains: lower write latency, tolerance of datacentre and network outages. The big drawback: the same data can be changed concurrently in two places, so you must resolve **write conflicts**, and features like auto-increment keys, triggers and integrity constraints become hazardous. → *More:* Ch 5 — Replication

Q75. ⋆ How do you resolve write conflicts in multi-leader or leaderless systems? Best: **avoid** them by routing all writes for a given record to the same leader (for example a user's "home" datacentre). Otherwise converge: **last write wins (LWW)** — keep the write with the highest timestamp, which silently loses data; merge the values (for example union of sets, or concatenation); record the conflict and let application code or the user resolve it later (on write or on read, as CouchDB does); or use **CRDTs** (Q77). Whatever the method, every replica must reach the same final value. → *More:* Ch 5 — Replication

Q76. Why is "last write wins" dangerous? LWW picks the write with the largest timestamp and throws the others away, even though the client was told they succeeded — silent data loss. Worse, the timestamps come from unsynchronised clocks: a node whose clock is ahead can make its writes "win" over writes that actually happened later. LWW is safe only when each key is written once and never updated (for example, using a UUID as the key for each new write). Cassandra uses LWW by default. → *More:* Ch 5 — Replication

Q77. What is a CRDT? A **CRDT** (conflict-free replicated data type) is a data structure designed so that concurrent updates on different replicas can always be merged automatically into the same result, in any order. Examples: G-Counter (each replica counts its own increments; the value is the sum), OR-Set (add/remove with unique tags), sequence CRDTs for collaborative text (as in Automerge and Yjs). Riak has built-in CRDT types; Redis Enterprise uses them for active-active geo-replication. The cost is extra metadata and limited operations. → *More:* Ch 5 — Replication

Q78. ⋆ How does leaderless (Dynamo-style) replication work? There is no leader: the client (or a coordinator node) sends each write to all *n* replicas and reads from several replicas in parallel. Nodes that were down catch up via **read repair** (a reader that sees a stale value writes the newer one back) and **anti-entropy** (a background process compares replicas, often using Merkle trees, and copies missing data). The design comes from Amazon's Dynamo paper and is used by Cassandra, ScyllaDB and Riak. (Amazon DynamoDB, the cloud service, actually uses leader-based replication per partition.) → *More:* Ch 5 — Replication

Q79. ⋆ Explain quorums: why does w + r > n give up-to-date reads? With *n* replicas, a write must be acknowledged by *w* nodes and a read must query *r* nodes. If **w + r > n**, the set of nodes written and the set read must overlap in at least one node, so at least one of the values read is the latest. A typical choice is n = 3, w = 2, r = 2, which tolerates one unavailable node. Smaller w or r means lower latency and higher availability but more stale reads; w = n, r = 1 makes reads fast but any one dead node blocks writes. → *More:* Ch 5 — Replication

Q80. ⋆ Why can quorum reads still return stale data even with w + r > n? Several edge cases: a **sloppy quorum** may write to nodes outside the usual *n*; two concurrent writes may be resolved with LWW and one lost; a write concurrent with a read may be seen by some replicas only; a write that failed on most nodes is not rolled back on the ones where it succeeded; a node holding a new value fails and is restored from a replica with an old one; and timing edge cases with clocks. So Dynamo-style quorums do not give **linearizability** (Q135); treat w and r as tuning knobs for the probability of staleness, not hard guarantees. → *More:* Ch 5 — Replication

Q81. What are sloppy quorums and hinted handoff? During a network problem, a client may be unable to reach enough of a key's *n* "home" nodes. A **sloppy quorum** accepts the write on any *w* reachable nodes, even ones that do not normally hold that key. When the home nodes return, the stand-in nodes send them the writes — **hinted handoff**. It raises write availability, but a read of *r* home nodes may miss the latest value, so it is not a real quorum. Riak enables it by default; Cassandra has hinted handoff but does not count hints toward the consistency level (the only exception is consistency level `ANY`). → *More:* Ch 5 — Replication

Q82. How do you detect concurrent writes? What is a version vector? Two operations are **concurrent** if neither knew about the other (not about wall-clock overlap). A server keeps a **version number** per key, incremented on each write; a client must read before writing and send back the version it read. A write that includes version *v* overwrites everything up to *v* but keeps later values as concurrent **siblings** for the application to merge. With several replicas you need one counter per replica — a **version vector** —

and comparing two vectors tells you whether one supersedes the other or they are concurrent. (Vector clocks are a closely related idea.) → *More:* Ch 5 — Replication

Q83. How do you deal with deletions when merging siblings (e.g. a shopping cart)? If you merge siblings by taking the union of their items, an item removed in one sibling comes back from the other. So a deletion must leave a **tombstone** — a marker with a version saying "this item was removed" — so the merge knows to drop it. This is exactly the problem CRDTs (such as the OR-Set) solve automatically. The Dynamo paper reported deleted items reappearing in shopping carts for this reason. → *More:* Ch 5 — Replication

Q84. In MongoDB, what do write concern and read concern control? Write concern says how many replica-set members must acknowledge a write before it succeeds: `w: 1` (primary only), `w: "majority"` (the implicit default since MongoDB 5.0, except in some replica sets with arbiters, where it falls back to `w: 1`), plus `j: true` to wait for the journal. **Read concern** sets what data a read may see: `local` (whatever this node has, possibly rolled back later), `majority` (only majority-committed data, never rolled back), `linearizable`. Together with **read preference** (primary vs secondaries) they are MongoDB's version of the quorum and replication-lag trade-offs in this chapter. → *More:* Ch 5 — Replication

Partitioning (sharding)

Q85. ⋆ What is partitioning (sharding) and why do we need it? Partitioning splits a large dataset into smaller pieces called partitions (shards in MongoDB/Elasticsearch, regions in HBase, tablets in Bigtable, vnodes in Cassandra), each stored on a different node. The main reason is **scalability**: data volume and query load are spread across many machines, beyond what one machine can hold. Each record belongs to exactly one partition, and partitioning is usually combined with replication, so each partition is copied on several nodes. → *More:* Ch 6 — Partitioning

Q86. ⋆ Range partitioning vs hash partitioning? Key-range partitioning assigns a contiguous range of keys to each partition (like encyclopedia volumes A–B, C–D…). Range scans are efficient, but some access patterns create **hot spots** — for example, timestamp keys send all of today's writes to one partition. **Hash partitioning** applies a hash function to the key and assigns ranges of hashes; load spreads evenly, but range queries must hit every partition. Cassandra combines both with a **compound primary key**: the first part is hashed to pick the partition, and the rest sorts the rows within it. → *More:* Ch 6 — Partitioning

Q87. ⋆ What is a hot spot / skew and how do you fix a hot key? Skew means some partitions get more data or queries than others; a partition with very high load is a **hot spot**. Even hashing cannot fix one hot key (a celebrity's posts all share one user ID). Fixes: add a random suffix (say 00–99) to the hot key so its writes spread over 100 partitions — reads must then query all 100 and merge, so only do this for the few keys that need it and track which keys are split; or cache hot reads in front of the database. Also prefix timestamp keys with something like a sensor name to spread time-ordered writes. → *More:* Ch 6 — Partitioning

Q88. ⋆ What is consistent hashing? Consistent hashing maps both nodes and keys onto a hash ring; each key belongs to the next node clockwise. Adding or removing a node only moves the keys between it and its neighbour, instead of reshuffling nearly everything as `hash(key) mod N` does. **Virtual nodes** (each physical node owns many small points on the ring) smooth out the load and let a new node take a little data from every other node. Used by Dynamo-style stores (Cassandra, Riak), memcached client libraries and CDNs. The book notes the term is used loosely, and many databases actually use fixed partitions or ranges of hashes. → *More:* Ch 6 — Partitioning

Q89. Why is "hash mod N" a bad way to assign partitions to nodes? When N changes (you add a node), almost every key maps to a different node — with mod 10 → mod 11, about 90% of keys move. That is a massive, unnecessary data transfer. Better strategies move only the minimum data: a fixed number of partitions, dynamic partitioning, or consistent hashing. → *More:* Ch 6 — Partitioning

Q90. ⋆ Compare the rebalancing strategies: fixed partitions, dynamic partitioning, partitioning proportional to nodes. Fixed number of partitions: create many more partitions than nodes (say 1,000 for 10 nodes); a new node steals whole partitions from others; key-to-partition mapping never changes (Elasticsearch, Riak, Couchbase). Hard part: choosing the count up front. **Dynamic partitioning**: split a partition when it grows past a size (HBase 10 GB default) and merge small ones; adapts to data volume,

works with ranges (HBase, MongoDB). **Proportional to nodes**: a fixed number of partitions per node (Cassandra's tokens per node), so partition size grows with data and shrinks when nodes are added. → *More:* Ch 6 — Partitioning

Q91. ⋆ How do secondary indexes work with partitioned data? Local vs global indexes? A **local (document-partitioned) index** — each partition indexes only its own documents. Writes touch one partition, but a query by the indexed field must ask *every* partition and combine the results (**scatter/gather**), which suffers from tail latency (MongoDB, Cassandra, Elasticsearch). A **global (term-partitioned) index** — the index itself is partitioned by the indexed term (colour=red lives on one partition). Reads go to one index partition, but a single write may update several index partitions, so global indexes are usually updated **asynchronously** (DynamoDB's global secondary indexes). → *More:* Ch 6 — Partitioning

Q92. ⋆ How does a client know which node holds a given key (request routing)? Three approaches: (1) the client contacts any node, which forwards the request if needed (Cassandra and Riak, using a **gossip protocol** to share cluster state); (2) a **routing tier** that knows the partition map forwards requests (MongoDB's mongos); (3) clients know the assignment themselves. Someone must hold the authoritative map — often a coordination service like **ZooKeeper** (HBase, SolrCloud, older Kafka) or a Raft-based store (Kafka's built-in KRaft controllers; etcd, which Kubernetes uses) that notifies routers when partitions move. → *More:* Ch 6 — Partitioning

Q93. Should rebalancing be automatic or manual? Fully automatic rebalancing is convenient but risky: rebalancing moves lots of data, and if an overloaded node is wrongly judged dead, the automatic system may move load onto other nodes and cause a **cascading failure**. Many systems (Couchbase, Riak, Voldemort) generate a suggested plan automatically but require an administrator to commit it. A human in the loop is slower but avoids surprises. → *More:* Ch 6 — Partitioning

Q94. ⋆ How would you choose a shard key? A good shard key has high **cardinality** (many distinct values), spreads writes evenly (no monotonic timestamps or auto-increment IDs as a range key), and matches your main query pattern so most queries hit one shard (shard a multi-tenant SaaS by `tenant_id`, chat by `conversation_id`). Watch for very large tenants (split them further), cross-shard queries and transactions (keep data that is transacted together in one shard), and the cost of ever changing the key — resharding is a large migration. → *More:* Ch 6 — Partitioning

Q95. How do you run a query or join that spans many shards? For simple key lookups, route to one shard. For queries by a non-shard field, use scatter/gather over local indexes or a global index. Big analytic queries use **massively parallel processing (MPP)**: the query optimizer splits the query into stages that run in parallel on each partition, then combines results (Teradata, Redshift, Spark SQL, Trino). For joins, either co-partition both tables on the join key or broadcast the small table. If you need this often for OLTP, reconsider the shard key or copy data into an analytics store. → *More:* Ch 6 — Partitioning

Q96. Partitioning vs replication — how do they combine? They are independent: partitioning decides *which node holds which part* of the data, replication decides *how many copies* of each part exist. Usually each partition is replicated to several nodes, and each node holds several partitions — being the leader for some and a follower for others — so the load and the failure impact are spread out. → *More:* Ch 6 — Partitioning

Q97. How do you generate unique IDs across shards? Options: **UUIDs** (random 128-bit, no coordination, but random v4 UUIDs scatter inserts across a B-tree; time-ordered **UUIDv7** fixes that — PostgreSQL 18 added a built-in `uuidv7()` function); **Snowflake-style IDs** (64 bits = timestamp + machine ID + per-machine sequence; roughly time-sorted, compact, but depend on clocks); a central ticket server or database sequence (simple but a bottleneck/single point of failure); or per-shard ranges handed out in blocks. Choose based on whether you need ordering, size and coordination-free generation. → *More:* Ch 0a — System Design Fundamentals

Q98. How would you reshard a live database with no downtime? Typical plan: create the new shards; start **dual writes** or (better) stream changes from the old shards to the new ones using change data capture; **backfill** existing data with a snapshot plus the change stream from the snapshot's position; verify with checksums or comparison reads; move reads to the new shards gradually, then writes (often per tenant or key range, with a short write pause or a routing switch); keep the old shards for rollback. Tools like Vitess and Citus automate parts of this. Call out the hard parts: ordering between the backfill and live changes, and the cutover moment. → *More:* Ch 6 — Partitioning

Transactions and isolation anomalies

Q99. ⋆ What does ACID stand for, and what does each letter really mean? Atomicity: all writes in a transaction happen, or none do — if it fails midway, the database undoes the partial writes (it is really about *abortability*, not concurrency). **Consistency**: the application's invariants (for example, debits equal credits) hold — mostly the application's job, the database only enforces declared constraints. **Isolation**: concurrent transactions do not interfere; ideally the result is as if they ran one after another (serializability), though most databases default to weaker levels. **Durability**: once committed, data is not lost — written to disk (WAL + fsync) and/or replicated. The book notes "ACID" has become partly a marketing term, so ask what exactly a system guarantees. → *More:* Ch 7 — Transactions

Q100. What is BASE? BASE — Basically Available, Soft state, Eventual consistency — was coined as the opposite of ACID for systems that relax guarantees for availability and scale. In practice it only says "not ACID", so it is even vaguer than ACID. In an interview, say what each specific guarantee is rather than relying on either acronym. → *More:* Ch 7 — Transactions

Q101. Why would you need a multi-object transaction? Isn't single-row atomicity enough? Single-object operations (an atomic increment, a compare-and-set on one row) are enough for some tasks, and most stores provide them. But you need multi-object transactions when several pieces of data must stay in sync: a foreign key between two rows, a denormalized counter that must match the rows it counts, or secondary indexes that must match the table. Without them, a crash or a concurrent reader can see half-done changes. Many NoSQL systems dropped multi-object transactions for scale and then added them back (MongoDB 4.0+, DynamoDB TransactWriteItems). → *More:* Ch 7 — Transactions

Q102. Is it safe to just retry an aborted transaction? Usually, but not always. If the transaction actually committed and only the acknowledgment was lost, a retry runs it twice (unless you add idempotency/deduplication). If the error was overload, retrying makes it worse (use exponential backoff and a retry limit). Retry only transient errors (deadlock, serialization failure, timeout), not permanent ones (constraint violation). And side effects outside the database — sending an email — happen again on retry unless they are made part of an atomic commit or deduplicated. → *More:* Ch 7 — Transactions

Q103. ⋆ Name the isolation levels and the anomalies each one prevents.

Level	Dirty read	Dirty write	Non-repeatable read / read skew	Lost update	Phantom / write skew
Read uncommitted	possible	prevented	possible	possible	possible
Read committed	prevented	prevented	possible	possible	possible
Snapshot isolation (often called repeatable read)	prevented	prevented	prevented	depends on DB	write skew possible
Serializable	prevented	prevented	prevented	prevented	prevented

Defaults: PostgreSQL, Oracle and SQL Server use read committed; MySQL InnoDB uses repeatable read. Names differ between databases, so describe the anomaly, not just the level name. → *More:* Ch 7 — Transactions

Q104. ⋆ What is a dirty read and a dirty write? A **dirty read** is seeing another transaction's uncommitted data — which may then be rolled back, so you acted on data that never existed. A **dirty write** is overwriting another transaction's uncommitted value — for example two people buying the same car, where the listing goes to one buyer and the invoice to the other. Read committed prevents both: dirty writes with row-level locks held until commit; dirty reads by giving readers the last committed value (not by read locks, which would make readers wait behind long writes). → *More:* Ch 7 — Transactions

Q105. ⋆ What is read skew (non-repeatable read)? Give an example. Read skew is reading different parts of the database at different points in time. Example: Alice has $500 in each of two accounts; a transfer of $100 commits between her two reads, so she sees $500 in one and $400 in the other — $900 total, as if $100 vanished. Refreshing fixes it for her, but it is fatal for **backups** (a backup mixing

old and new parts) and **long analytic queries or integrity checks**. Snapshot isolation prevents it. → *More:* Ch 7 — Transactions

Q106. ⋆ What is snapshot isolation and how does MVCC implement it? Snapshot isolation lets each transaction read from a consistent **snapshot** of the database as it was when the transaction started, even while others change the data. **MVCC** (multi-version concurrency control) implements it by keeping several committed versions of each row, each tagged with the transaction ID that created it (and the one that deleted it). A transaction sees a version only if its creator committed before the transaction started and it was not deleted by such a transaction. Key property: **readers never block writers and writers never block readers**. Used by PostgreSQL, MySQL InnoDB, Oracle, SQL Server (optional). → *More:* Ch 7 — Transactions

Q107. Why does PostgreSQL need VACUUM? Under MVCC, an `UPDATE` writes a new row version and a `DELETE` just marks the old one as deleted; old versions stay in the table. **VACUUM** (normally autovacuum) removes versions no running transaction can see any more, so the space can be reused, and it also prevents **transaction-ID wraparound** (IDs are 32-bit). A long-running or idle-in-transaction session holds back the cleanup horizon and causes **table bloat**. MySQL InnoDB keeps old versions in its undo log instead and purges them in the background. → *More:* Ch 7 — Transactions

Q108. ⋆ What is a lost update? How do you prevent it? A **lost update** happens with a **read-modify-write cycle**: two transactions read the same value (counter = 42), each computes a new value (43), and the second write overwrites the first — one increment is lost. Prevention: **atomic operations** (`UPDATE counters SET value = value + 1`); **explicit locking** (`SELECT … FOR UPDATE` , then update); **automatic detection** — PostgreSQL's repeatable read and Oracle's serializable abort one of the transactions (MySQL InnoDB's repeatable read does not); **compare-and-set** (`UPDATE … SET v = new WHERE id = 1 AND v = old` , check rows affected = 1); in replicated systems, siblings plus merge or CRDTs. → *More:* Ch 7 — Transactions

Q109. ⋆ Optimistic vs pessimistic locking — when to use each? Pessimistic locking assumes conflicts: lock the row before you change it (`SELECT … FOR UPDATE`), others wait. Good when conflicts are frequent or retries are expensive. **Optimistic** locking assumes conflicts are rare: read a version number, then write with `WHERE version = ?` (or let the database detect conflicts at commit, as SSI does); on conflict, abort and retry. Good for low contention and long user "think time", where holding locks would be wasteful. Under high contention optimistic schemes waste work on retries.

```
UPDATE account SET balance = 80, version = version + 1
WHERE id = 7 AND version = 3;   -- 0 rows updated => someone else won, retry
```

→ *More:* Ch 7 — Transactions

Q110. ⋆ What is write skew? Give the on-call doctors example. Write skew happens when two transactions read the same data, each decides something based on it, and then they write to *different* rows — so neither overwrites the other, yet together they break an invariant. Example: the rule "at least one doctor on call". Alice and Bob are both on call and both feel sick; each transaction checks "are at least 2 doctors on call?" (yes), then sets *their own* row to off call. Both commit, and now no one is on call. Snapshot isolation does not prevent this, because the transactions update different objects. Other examples: double-booking a meeting room, two users claiming the same username, double-spending. → *More:* Ch 7 — Transactions

Q111. ⋆ What is a phantom? How do you prevent write skew without full serializability? A **phantom** is when a write in one transaction changes the result of a search query in another — typically, a row that did not exist yet when you checked "are there any bookings for this room at 12:00?". You cannot lock a row that does not exist with `SELECT … FOR UPDATE` . Options: lock the rows the check depends on (when they exist, like the doctors' rows); **materialize the conflict** — create a table of rows to lock (for example one per room per 15-minute slot) — an ugly last resort; add a **unique or exclusion constraint** (PostgreSQL `EXCLUDE USING gist (room_id WITH =, during WITH &&)` for overlapping time ranges; the `=` on a plain column needs the `btree_gist` extension); or simply use serializable isolation. → *More:* Ch 7 — Transactions

Q112. ⋆ What are the three ways to implement serializability?

1. **Actual serial execution**: run transactions one at a time on a single thread (VoltDB/H-Store, Redis, Datomic) — works if transactions are short, data fits in memory and are submitted as **stored procedures** (no interactive back-and-forth). 2. **Two-phase locking (2PL)**: readers and writers block

each other with shared and exclusive locks (MySQL InnoDB and SQL Server serializable). 3. **Serializable snapshot isolation (SSI)**: snapshot isolation plus detection of dangerous conflicts at commit (PostgreSQL serializable since 9.1, CockroachDB, FoundationDB). → *More:* Ch 7 — Transactions

Q113. ⋆ How does two-phase locking (2PL) work? Is it the same as two-phase commit? No — 2PL is about **isolation** on one database, 2PC is about **atomic commit** across several nodes (Q150). In 2PL, a transaction takes a **shared lock** to read and an **exclusive lock** to write; a writer waits for all readers and readers wait for writers. Locks are held until the transaction ends — the "two phases" are acquiring locks while running and releasing them all at the end. Phantoms are prevented with **predicate locks** or, in practice, **index-range (next-key) locks** that lock a range of the index. Costs: much lower throughput, unpredictable latency, and frequent **deadlocks** that the database detects and resolves by aborting one transaction. → *More:* Ch 7 — Transactions

Q114. ⋆ How does serializable snapshot isolation (SSI) work? SSI is **optimistic**: transactions run on a snapshot without blocking each other, and the database tracks what each one read and wrote. At commit it checks whether the transaction's premise is still valid: did its snapshot ignore another transaction's write that was still uncommitted at the time and has since committed (**stale MVCC read**), or did it read data that a concurrent transaction later wrote (**writes that affect prior reads**)? If such a dangerous pattern of dependencies is found, one transaction is aborted with a serialization failure and must be retried. It avoids 2PL's blocking, but high contention causes many aborts, so keep read-write transactions short. → *More:* Ch 7 — Transactions

Q115. What is the difference between serializability and linearizability? Serializability is an *isolation* property of *transactions* over many objects: the result equals *some* serial order, not necessarily the real-time order. **Linearizability** is a *recency* guarantee on *single objects*: once a write completes, every later read sees it, matching real time. A database providing both is **strict serializable** (strong one-copy serializability): implementations based on actual serial execution or 2PL typically are; SSI is not linearizable by design, because it reads from a snapshot. → *More:* Ch 9 — Consistency and Consensus

Q116. What does `SELECT … FOR UPDATE` do, and what are its pitfalls? It reads rows and takes exclusive (write) locks on them, so other transactions trying to lock or update them wait until you commit — preventing lost updates in a read-modify-write. Pitfalls: it cannot lock rows that do not exist yet (phantoms, unless the database uses next-key locks); locking rows in different orders causes deadlocks (always lock in a consistent order); and holding locks while waiting on the user or a network call kills throughput. `SKIP LOCKED` and `NOWAIT` variants help build job queues and fail-fast behaviour. → *More:* Ch 7 — Transactions

Q117. What is a deadlock in a database and how is it handled? A **deadlock** is two (or more) transactions each waiting for a lock held by the other, so none can continue. Databases detect the cycle in the waits-for graph (or use a timeout) and abort one transaction as the victim; the application must retry it. Reduce them by keeping transactions short, touching rows in a consistent order, and using the right indexes so fewer rows are locked. → *More:* Ch 7 — Transactions

Q118. ⋆ How would you implement a money transfer between two accounts correctly? In one database: one transaction that debits and credits, with a check that the balance does not go negative — `UPDATE accounts SET balance = balance - 100 WHERE id = A AND balance >= 100` (check 1 row affected), then credit B, commit; lock rows in ID order to avoid deadlocks. Better still, store transfers as an append-only **ledger** of entries and derive balances. Across services or databases, avoid distributed transactions: write the transfer request with a unique ID, process it idempotently in each step, and use a saga (Q187) or a log-based approach (Q189). Always make the API idempotent with a client-supplied request ID. → *More:* Ch 7 — Transactions

Q119. Why is "repeatable read" confusing across databases? The SQL standard's definition is based on the 1975 System R design and does not describe snapshot isolation, so vendors use the name differently. PostgreSQL's repeatable read is snapshot isolation (and detects lost updates). MySQL InnoDB's repeatable read uses a snapshot for plain reads but locking reads see the latest data, and it does not detect lost updates. Oracle calls its snapshot isolation "serializable". IBM DB2 uses "repeatable read" for serializability. Always ask which anomalies are actually prevented. → *More:* Ch 7 — Transactions

Q120. Why do stored procedures matter for serial execution, and what are their downsides? With one thread executing transactions, an interactive transaction (waiting for the app to send the next query

over the network) would stall everything. So serial-execution systems require the whole transaction to be submitted up front as a **stored procedure**. Downsides: vendor-specific languages (though VoltDB uses Java, Redis uses Lua), harder to debug, version and test, and a badly written procedure can hurt the whole database. To scale, the data is partitioned so each CPU core runs its own partition serially — but cross-partition transactions are much slower. → *More:* Ch 7 — Transactions

The trouble with distributed systems

Q121. ⋆ What is a partial failure, and why does it make distributed systems hard? On a single computer, things usually either work or crash completely. In a distributed system, some parts can be broken in unpredictable ways while others work fine — a **partial failure**. Worse, partial failures are **nondeterministic**: a request may or may not have succeeded, and you may not even know. So distributed software must assume faults and build reliability from unreliable parts. → *More:* Ch 8 — The Trouble with Distributed Systems

Q122. ⋆ You send a request and get no response. What could have happened? The request may have been lost; it may be waiting in a queue; the remote node may have crashed or paused (for example a long garbage-collection pause); the node may have processed the request but the response was lost or is delayed. The sender cannot tell these apart. The usual tool is a **timeout**, but after one you still do not know whether the action happened — which is why retries need **idempotency**. → *More:* Ch 8 — The Trouble with Distributed Systems

Q123. ⋆ How do you choose a timeout for failure detection? Too long: users wait and the system is down for longer before it fails over. Too short: a node that is merely slow (load spike, GC pause) is declared dead, its work moves to others, adding load that can cascade into more false "deaths" — and its action may run twice. Network delays are unbounded in practice (queueing in switches, OS, VMs, TCP retransmissions), so measure the distribution of round-trip times and set timeouts adaptively, as Cassandra and Akka do with the **Phi accrual failure detector**, which outputs a suspicion level rather than a yes/no. → *More:* Ch 8 — The Trouble with Distributed Systems

Q124. Why are datacentre networks asynchronous rather than synchronous like telephone circuits? A telephone call reserves fixed bandwidth along the whole path (a **circuit**), so delay is bounded. Internet and datacentre networks use **packet switching** optimised for **bursty** traffic: no reservation, packets share links and queue when links are busy, so delay is unbounded but utilisation is high. Bounded delay is possible (circuits, quality-of-service, admission control) but costs utilisation and money, so we accept variable delays and design for them. → *More:* Ch 8 — The Trouble with Distributed Systems

Q125. ⋆ Time-of-day clock vs monotonic clock — which do you use for measuring a duration? A **time-of-day (wall) clock** (`System.currentTimeMillis()` , `clock_gettime(CLOCK_REALTIME)`) returns the current date and time; NTP can step it forward or *backward*, so differences between two readings are unreliable. A **monotonic clock** (`System.nanoTime()` , `CLOCK_MONOTONIC`) always moves forward; only differences on the same machine are meaningful. So: monotonic for timeouts and durations, wall clock only when you need an actual date — and never compare wall-clock timestamps across machines for ordering. → *More:* Ch 8 — The Trouble with Distributed Systems

Q126. ⋆ Why can't we rely on synchronized clocks to order events across machines? Quartz clocks drift (Google assumes 200 ppm — about 17 seconds per day if not resynced); NTP synchronisation is limited by network delay (tens of milliseconds over the internet), can be misconfigured or firewalled, and steps clocks; VMs pause; leap seconds cause bugs. So two events on different nodes can get timestamps that contradict the order in which they really happened, and LWW then silently drops the later write. Use **logical clocks** (Lamport timestamps, version vectors) to order events instead. → *More:* Ch 8 — The Trouble with Distributed Systems

Q127. How does Google Spanner use TrueTime to get consistent snapshots? TrueTime returns an interval `[earliest, latest]` that is guaranteed to contain the real current time, using GPS receivers and atomic clocks in every datacentre to keep the uncertainty around 7 ms. When committing a read-write transaction, Spanner picks a timestamp and then deliberately **waits out the uncertainty** ("commit wait") before making the result visible. That guarantees that if transaction A finished before B started, A's timestamp is lower — so timestamps match real-time order and snapshots across datacentres are

consistent. The smaller the uncertainty, the shorter the wait. → *More:* Ch 8 — The Trouble with Distributed Systems

Q128. ⋆ What are process pauses, and why are they dangerous for leases and locks? A thread can be paused for seconds at any point: garbage-collection "stop-the-world" pauses, a VM being suspended or live-migrated, a laptop lid closed, OS context switches, swapping to disk, a `SIGSTOP`. The process does not notice. Danger: a node checks "my lease is valid for 10 more seconds", then pauses for 15 seconds, wakes up and writes as if it were still the leader — while another node has already taken over. You cannot fix this with more careful checking; you need fencing tokens (Q129). → *More:* Ch 8 — The Trouble with Distributed Systems

Q129. ⋆ What is a fencing token? Why isn't a distributed lock with a timeout safe on its own? A **fencing token** is a number that increases every time the lock service grants a lock or lease. Every write to the protected storage must include it, and the storage rejects any write with a lower token than one it has already seen. So if client 1 pauses, its lease expires, client 2 gets token 34 and writes, and client 1 wakes up and writes with token 33, the storage rejects it. A lock with a timeout alone (for example a basic Redis lock) cannot stop a paused client from writing after expiry. ZooKeeper's `zxid` or node version can serve as the token. → *More:* Ch 8 — The Trouble with Distributed Systems

Q130. What is a Byzantine fault? Do typical backend systems need to tolerate them? A **Byzantine fault** is a node that behaves arbitrarily — sending corrupted or contradictory messages, or lying on purpose. Byzantine fault-tolerant protocols (needing more than two-thirds of nodes honest) are used in aerospace and blockchains with mutually untrusting parties. Typical datacentre systems assume nodes are honest but may crash or be slow, because all nodes are controlled by one organisation; BFT would be too costly. Still, guard against weak forms of lying: checksums for corrupted packets, input validation for user data, multiple NTP servers. → *More:* Ch 8 — The Trouble with Distributed Systems

Q131. What system models do distributed algorithms assume? Timing models: **synchronous** (bounded delays, pauses and clock error — unrealistic), **partially synchronous** (usually behaves synchronously, sometimes exceeds bounds — the realistic one), **asynchronous** (no timing assumptions, no clocks at all). Node failure models: **crash-stop** (a node that fails is gone forever), **crash-recovery** (nodes may come back with their durable storage intact; the usual one), **Byzantine** (arbitrary). Algorithms are proved correct within a model; real systems also need to handle cases the model leaves out. → *More:* Ch 8 — The Trouble with Distributed Systems

Q132. What is the difference between safety and liveness properties? A **safety** property says nothing bad ever happens (for example, uniqueness of fencing tokens: no two clients get the same token); if violated, you can point to the moment it broke, and the damage cannot be undone. A **liveness** property says something good eventually happens (for example, a request eventually gets a response); it may not hold right now but could later. Distributed algorithms usually require safety to hold always, even if every node crashes, while liveness is only guaranteed under some conditions (for example a majority of nodes up and the network recovered). → *More:* Ch 8 — The Trouble with Distributed Systems

Q133. "The truth is defined by the majority" — what does that mean? A single node's view of the world can be wrong: after a pause or a network cut, it may still believe it leads while everyone else has already written it off. So decisions — who is leader, whether a node is dead, whether a write is committed — are made by a **quorum**, usually a majority of nodes. A majority is used because there can be only one majority at a time, so two conflicting decisions cannot both win. A node that the quorum declares dead must step down even if it feels fine. → *More:* Ch 8 — The Trouble with Distributed Systems

Q134. What is a network partition, and how common are they really? A **network partition** (netsplit) is when part of the network is cut off from the rest while the nodes on each side keep running. Studies cited in the book found frequent network faults even in carefully managed datacentres — about 12 per month in one mid-sized datacentre, misconfigured switches, even sharks biting undersea cables. So your software must have a defined behaviour during a partition, and you should test it (for example with Jepsen-style fault injection). Note a partition is different from a node crash: both sides keep running and may keep acting. → *More:* Ch 8 — The Trouble with Distributed Systems

Consistency and consensus

Q135. ⋆ What is linearizability? Linearizability (atomic consistency, strong consistency) makes a replicated system behave as if there were **only one copy of the data** and every operation took effect atomically at some instant between its start and end. Its key **recency guarantee**: once any read has returned a new value (or a write has completed), all later reads — by anyone — return that value or a newer one, never an older one. It is what you need for leader election, locks, uniqueness constraints and "is this seat still free" checks. → *More:* Ch 9 — Consistency and Consensus

Q136. ⋆ Which replication methods can be linearizable? Single-leader: potentially, if you read from the leader (or synchronously updated followers) and know for sure who the leader is — but split brain or async failover breaks it. **Consensus algorithms** (ZooKeeper, etcd): yes, they are designed for it (with care on reads). **Multi-leader**: no — concurrent writes in several places. **Leaderless (Dynamo-style)**: probably not; LWW with clocks breaks it, and even strict quorums with w + r > n can show non-linearizable behaviour unless readers do synchronous read repair and writers read a quorum first (and even then compare-and-set needs consensus). → *More:* Ch 9 — Consistency and Consensus

Q137. ⋆ Explain the CAP theorem. Why is "pick two of three" misleading? CAP says that when a **network partition** happens, a system must choose between **consistency** (here meaning linearizability) and **availability** (every request to a non-failed node gets a non-error response). Without a partition you can have both. "Pick 2 of 3" is misleading because partitions are not optional — they happen whether you like it or not — so the real choice is "**C or A when partitioned**". The book also points out CAP's definitions are narrow (one consistency model, one kind of fault) and it says nothing about latency or crashed nodes, so it is of little practical help for design; it mattered historically for opening up the design space. → *More:* Ch 9 — Consistency and Consensus

Q138. What is PACELC? PACELC extends CAP: if there is a **P**artition, choose **A**vailability or **C**onsistency; **E**lse (normal operation), choose **L**atency or **C**onsistency. It captures the cost CAP ignores — even without failures, strong consistency costs latency because replicas must coordinate. Examples: Cassandra and Dynamo-style stores are PA/EL; Spanner and traditional synchronous systems are PC/EC; MongoDB is often described as PA/EC by default. → *More:* Ch 0a — System Design Fundamentals

Q139. Why do many systems choose not to be linearizable even without partitions? Performance. Linearizability requires coordination, so response time is at least proportional to the uncertainty of network delays — Attiya and Welch proved this. The same reasoning applies inside one machine: CPU cores work against their own caches, so shared memory is not linearizable unless code adds memory barriers. So systems give it up for speed; partitions are a secondary concern. → *More:* Ch 9 — Consistency and Consensus

Q140. ⋆ Strong vs eventual vs causal consistency — explain the spectrum. Eventual consistency: if writes stop, replicas eventually converge; until then, any read may be stale and reads may go back in time. **Causal consistency**: operations that are causally related (a reply after the message it answers; a write after the read it was based on) are seen by everyone in that order, while concurrent operations may be seen in different orders. **Linearizability (strong)**: a single total order matching real time. Causal consistency is the strongest model that stays available during network partitions, making it an attractive middle ground; session guarantees (read-your-writes, monotonic reads) are pieces of it. → *More:* Ch 9 — Consistency and Consensus

Q141. ⋆ What are Lamport timestamps? How are they different from vector clocks? A **Lamport timestamp** is a pair (counter, node ID). Each node increments its counter on each event, and every message carries the sender's maximum counter; a receiver that sees a bigger counter jumps its own counter forward to it. Ordering by (counter, then node ID) gives a **total order consistent with causality**: if A happened before B, A's timestamp is smaller. But from two timestamps you cannot tell whether events were concurrent or causally related. **Vector clocks / version vectors** keep one counter per node and *can* detect concurrency, at the cost of more metadata. → *More:* Ch 9 — Consistency and Consensus

Q142. Why are Lamport timestamps not enough to enforce a uniqueness constraint? To decide "who gets username *alice*", a node needs to know it has seen *every* request with a smaller timestamp — but another node might have issued a smaller timestamp that has not arrived yet, so you would have to ask every node, and you are stuck if one is unreachable. The total order only becomes known *after the fact*. To

decide in real time you need **total order broadcast** (or consensus). → *More:* Ch 9 — Consistency and Consensus

Q143. What is total order broadcast and why is it equivalent to consensus? **Total order broadcast** (atomic broadcast) is a protocol for sending messages to all nodes with two guarantees: **reliable delivery** (if a message is delivered to one node, it is delivered to all) and **totally ordered delivery** (every node delivers messages in the same order). It is exactly what a replicated log needs: **state machine replication** — every replica applies the same deterministic writes in the same order. Solving total order broadcast lets you solve consensus (decide the first message in the log) and vice versa (run consensus for each log slot). ZooKeeper's Zab and etcd's Raft implement it. → *More:* Ch 9 — Consistency and Consensus

Q144. How can you build a linearizable compare-and-set (e.g. a unique username) on top of total order broadcast? Append a message "I claim username X" to the log; read the log until your message comes back; if the *first* claim for X in the log is yours, you got it — commit (and confirm to the user); otherwise abort. Every node sees the same order, so they all agree who won. Reads can be made linearizable by appending a "read" marker to the log and serving the read when it is delivered, or by waiting until you have caught up to the latest log position (etcd's quorum reads, ZooKeeper's `sync()`). → *More:* Ch 9 — Consistency and Consensus

Q145. ⋆ What is the consensus problem and where is it needed? **Consensus** means getting several nodes to agree on one value, with these properties: **uniform agreement** (no two nodes decide differently), **integrity** (no node decides twice), **validity** (the value decided was proposed by some node), and **termination** (every non-crashed node eventually decides). Needed for **leader election** (exactly one leader, no split brain), **atomic commit** (all nodes commit or all abort), **uniqueness constraints**, locks and leases, and **membership** (which nodes are alive). Consensus can tolerate up to f failures with $2f + 1$ nodes (a majority must be alive). → *More:* Ch 9 — Consistency and Consensus

Q146. ⋆ Explain how Raft works. Raft keeps a replicated log across (usually 3 or 5) servers. **Leader election**: time is split into numbered **terms**; a follower that hears no heartbeat within a randomized election timeout becomes a candidate, increments the term and asks for votes; a node votes for at most one candidate per term, and only for a candidate whose log is at least as up to date as its own; a majority of votes makes a leader. **Log replication**: the leader appends client commands to its log and sends them to followers (AppendEntries); an entry is **committed** once stored on a majority, then applied to the state machine. (Subtlety: a leader only counts replicas for entries of its *own* term; entries left over from earlier terms become committed indirectly when a newer entry after them commits.) A leader that sees a higher term steps down. Used by etcd, Consul, CockroachDB, TiKV, Kafka's KRaft and MongoDB's replica sets (a Raft-like protocol). → *More:* Ch 9 — Consistency and Consensus

Q147. Raft vs Paxos? Both solve consensus with majorities and an increasing number (Paxos **ballot/proposal numbers**, Raft **terms**; the book's general name is **epoch**). **Paxos** (Lamport) is the original; basic Paxos agrees on one value, Multi-Paxos on a log, and it is known for being hard to understand and leaving many practical details open. **Raft** (2014) was designed for understandability: a strong leader, explicit log-matching rules and a full spec including membership changes, so it became the common choice for new systems. **Zab** (ZooKeeper) and Viewstamped Replication are similar. All have the same core: a leader proposes, a quorum votes, and the epoch number prevents an old leader from overwriting newer decisions. → *More:* Ch 9 — Consistency and Consensus

Q148. Why do consensus clusters have an odd number of nodes? Consensus needs a **majority** to make progress. With 3 nodes the majority is 2, so 1 can fail; with 5, 2 can fail. A 4-node cluster also needs 3 for a majority and still tolerates only 1 failure — the extra node adds cost and latency but no fault tolerance (and in a 2+2 network split, neither side has a majority). So clusters use 3 or 5 nodes, rarely 7; more nodes make every write slower. → *More:* Ch 9 — Consistency and Consensus

Q149. What are the limits of consensus? Every decision needs a vote from a majority (synchronous replication), which costs latency. You need a strict majority alive, so a network partition can leave the minority side unable to work. Most algorithms assume a fixed set of nodes (dynamic membership is harder). Failure detection relies on timeouts, so in networks with highly variable delay you get frequent false leader elections, which can stall progress. Raft has also shown edge cases where one unreliable link causes leadership to bounce continuously. → *More:* Ch 9 — Consistency and Consensus

Q150. ⋆ How does two-phase commit (2PC) work? A **coordinator** (transaction manager) runs the commit across several **participants** (databases). **Phase 1 — prepare**: the coordinator asks every participant "can you commit?"; a participant that says *yes* must have made the writes durable and promises to commit if asked — it gives up the right to abort. **Phase 2 — commit/abort**: if all said yes, the coordinator writes the decision to its own log on disk (the **commit point**) and tells everyone to commit; if any said no or timed out, it tells everyone to abort. The coordinator retries the decision forever until every participant has received it. → *More:* Ch 9 — Consistency and Consensus

Q151. ⋆ What is the main problem with 2PC? What does "in doubt" mean? If the coordinator crashes after participants voted *yes* but before they hear the decision, those participants are **in doubt** (uncertain): they cannot commit or abort on their own, because the others might have done the opposite. They must hold their **row locks** and wait until the coordinator recovers — possibly for hours, blocking other transactions. So 2PC is a **blocking** atomic commit protocol, and the coordinator is effectively a single point of failure. If its log is lost, an admin may have to decide manually (**heuristic decisions**), which can break atomicity. → *More:* Ch 9 — Consistency and Consensus

Q152. Why doesn't three-phase commit (3PC) fix 2PC in practice? 3PC is non-blocking only if you assume bounded network delay and response times, so that a timeout reliably means a node has crashed. In real networks with unbounded delays, a timeout does not mean a crash, so 3PC cannot guarantee atomicity. A truly non-blocking atomic commit needs a perfect failure detector — which we do not have — or a consensus algorithm to replicate the coordinator's decision. → *More:* Ch 9 — Consistency and Consensus

Q153. Is 2PC the same as consensus? How do they differ? They are related but different. In 2PC, *every* participant must vote yes for commit — a single "no" or a single unreachable participant forces abort or blocking; consensus only needs a *majority*, and any proposed value can win. 2PC's coordinator is not elected and is a single point of failure; consensus algorithms elect a new leader automatically. Modern systems combine them: Spanner and CockroachDB run 2PC across shards where each participant (and the coordinator's state) is itself a Raft or Paxos group, so no single crash blocks the protocol. → *More:* Ch 9 — Consistency and Consensus

Q154. What are XA transactions and why are they unpopular? XA (X/Open eXtended Architecture) is a standard C API for 2PC across **heterogeneous** systems — for example a database and a message broker in one transaction (Java's JTA uses it). Problems: the coordinator often lives inside the application server, so its log becomes critical state and that server is a single point of failure; XA cannot detect deadlocks across systems and does not work with SSI; it is a lowest common denominator; and the performance cost is high (MySQL distributed transactions were reported as 10× slower). In-doubt transactions hold locks until resolved. So most teams prefer log-based integration and idempotency instead. → *More:* Ch 9 — Consistency and Consensus

Q155. ⋆ What does ZooKeeper (or etcd) actually provide? A small, fault-tolerant key-value store built on total order broadcast, designed to hold small amounts of coordination data in memory. Writes are linearizable; ZooKeeper serves reads from the local server, so they can be stale unless you call `sync()` first (etcd reads are linearizable by default). It offers: **linearizable atomic operations** (compare-and-set) for locks; **total ordering of operations** (the `zxid`, usable as a fencing token); **failure detection** via sessions and **ephemeral nodes** that disappear when a client's session dies; and **change notifications** (watches). Use cases: leader election, assigning partitions to nodes, service discovery, membership and configuration. You outsource consensus to it rather than implementing your own. Kafka 4.0 (2025) removed ZooKeeper and runs its own Raft-based controller quorum (KRaft) instead. → *More:* Ch 9 — Consistency and Consensus

Q156. How would you implement leader election in your service? Do not invent your own consensus. Use a coordination service: in ZooKeeper, every candidate creates an **ephemeral sequential** node under `/election`; the lowest sequence number is leader; each other candidate watches the node just ahead of it and checks again when it disappears. In etcd, use a lease plus a key with a transaction (or its built-in election API). In Kubernetes, use a `Lease` object. Then handle the hard part: the old leader may still be running after a pause, so pass a **fencing token** (term/epoch/zxid) with every write and have storage reject stale ones. → *More:* Ch 9 — Consistency and Consensus

Batch processing

Q157. ⋆ Services vs batch vs stream processing — what's the difference? Online services wait for a request and answer quickly; the measure is response time and availability. **Batch processing (offline)** runs a job over a large, **bounded** input (for example yesterday's logs) and produces output; the measure is **throughput**, and jobs typically run for minutes to days on a schedule. **Stream processing (near-real-time)** consumes **unbounded** input events shortly after they happen, with lower latency than batch. Batch and stream jobs produce **derived data** — data you could rebuild from the source of truth. → *More:* Ch 10 — Batch Processing

Q158. What is the Unix philosophy and how does MapReduce inherit it? Unix tools each do one thing well, read from stdin and write to stdout with a **uniform interface** (lines of text), don't modify their input, and can be composed with pipes (`cat log | awk … | sort | uniq -c | sort -rn | head`). Errors are easy to recover from because input is immutable. MapReduce is "Unix tools distributed across thousands of machines": mappers and reducers are composable functions, the distributed filesystem (HDFS) is the uniform interface, and input files are never modified. The difference: MapReduce writes intermediate output to files instead of streaming it through pipes. → *More:* Ch 10 — Batch Processing

Q159. ⋆ Explain how a MapReduce job works.

1. Input files on HDFS are split into blocks; the scheduler runs a **mapper** on each block, ideally on the machine that stores it (**putting the computation near the data**). 2. The mapper extracts key-value pairs from each record. 3. The **shuffle**: map output is partitioned by a hash of the key, sorted, and copied to **reducers**, so all values for one key arrive at the same reducer, in sorted order. 4. The reducer processes all values per key and writes the output to HDFS. Jobs are chained into **workflows** (Airflow, Oozie) where one job's output is the next one's input. → *More:* Ch 10 — Batch Processing

Q160. ⋆ How are joins done in batch processing? Sort-merge vs broadcast hash join? Reduce-side (sort-merge) join: mappers emit the join key from both datasets, the shuffle brings all records with the same key to one reducer (with a **secondary sort** so, say, the user record comes before their events), and the reducer joins them. Works on any data but is expensive (sorting and copying everything). **Map-side joins** avoid the shuffle when you know something about the data: **broadcast hash join** — the small dataset is loaded into an in-memory hash table on every mapper; **partitioned hash join** — both inputs are already partitioned the same way, so each mapper joins one matching pair of partitions; **map-side merge join** — both are also sorted. → *More:* Ch 10 — Batch Processing

Q161. How do you handle skew (a hot key) in a batch join? If one key has millions of records (a celebrity), the reducer that gets it takes far longer than the rest, and the job waits for it (a **straggler**). Fixes: **skewed join** (Pig) — sample to find the hot keys, spread their records over several reducers and send the other side of the join for those keys to all of them; Hive's skewed join stores hot keys separately and uses a map-side join for them; or a two-stage aggregation — first aggregate with a random suffix to spread the load, then combine. → *More:* Ch 10 — Batch Processing

Q162. Why should batch jobs write output to new files instead of writing directly into a production database? Writing one record at a time from thousands of tasks to a database is slow and can overload it, hurting online users; and if a task fails and reruns, the partial writes are already visible (side effects). Better: build a brand-new database file or index inside the job (like building Voldemort read-only stores, or a search index), write it to the distributed filesystem, then atomically switch the serving system to the new files. Output becomes immutable and reproducible — **human fault tolerance**: if you ship a bad job, rerun the fixed one or roll back to the old output. → *More:* Ch 10 — Batch Processing

Q163. Why did Spark (and Flink, Tez) replace MapReduce? MapReduce writes every intermediate result to HDFS (replicated) and every job must finish all mappers before reducers start, which wastes a lot of I/O and time in multi-step workflows. **Dataflow engines** (Spark, Flink, Tez) treat the whole workflow as one job — a graph of operators — keep intermediate state in memory or local disk, only sort when needed, schedule tasks with locality, and pipeline data between operators. Fault tolerance: Spark recomputes lost partitions from **lineage** (the recorded transformations of each RDD), Flink uses checkpoints. They also offer declarative high-level APIs (DataFrames, SQL) with a query optimizer. → *More:* Ch 10 — Batch Processing

Q164. How does a batch engine recover from a failed task, and why does determinism matter? A failed task is simply rerun on another machine from its input (MapReduce reruns from materialized intermediate files; Spark recomputes lost partitions from lineage). This only produces correct results if operators are **deterministic** — same input always gives same output. Nondeterminism (iterating a hash table in random order, using the current time or random numbers without a fixed seed) can make a recomputed partition differ from the original, so downstream tasks that already consumed the old version become inconsistent and must be rerun too. → *More:* Ch 10 — Batch Processing

Q165. Hadoop/data lake vs MPP database — what is the difference in philosophy? MPP databases (Teradata, Greenplum) require data to be loaded into a structured schema first and focus on running SQL fast. Hadoop takes the "**sushi principle**: raw data is better" approach — dump raw files of any format into the distributed filesystem quickly and decide how to interpret them later (schema-on-read), and run arbitrary code (machine learning, image processing), not just SQL. MapReduce also tolerates the failure of individual tasks at a fine granularity, which suits shared clusters where low-priority jobs get preempted. Today the two have converged (Spark SQL, Trino, lakehouses). → *More:* Ch 10 — Batch Processing

Q166. How are graph algorithms like PageRank run in batch? What is the Pregel model? Iterative graph algorithms repeat until convergence, which is very inefficient as a chain of MapReduce jobs (each iteration rereads the whole graph). The **Pregel (bulk synchronous parallel)** model: each vertex keeps state and in each iteration (**superstep**) processes messages sent to it and sends messages along edges; the framework handles partitioning, message delivery and checkpointing. Used in Apache Giraph, Spark GraphX and Flink Gelly. Cross-machine messages dominate the cost, so a graph that fits on one machine is often faster to process there. → *More:* Ch 10 — Batch Processing

Stream processing, CDC and event sourcing

Q167. ⋆ Message queue (RabbitMQ) vs log-based broker (Kafka) — what's the difference? A traditional **message broker** (RabbitMQ, ActiveMQ, Amazon SQS) assigns each message to a consumer and **deletes it once acknowledged**; consumers process in parallel without order guarantees across them, which is good for task queues where each message is expensive to process. A **log-based broker** (Kafka, Amazon Kinesis, Apache Pulsar) **appends** messages to a partitioned, persisted log; consumers read sequentially and track an **offset**; messages stay until retention expires, so you can **replay** them and many consumer groups read the same data independently. Order is guaranteed only within a partition, and parallelism is limited to the number of partitions. → *More:* Ch 11 — Stream Processing

Q168. ⋆ How does Kafka guarantee ordering, and how do you scale consumers? A Kafka **topic** is split into **partitions**; each partition is an ordered, append-only log. Messages with the same **key** go to the same partition, so events for one entity stay ordered. A **consumer group** shares the partitions: each partition is read by exactly one consumer in the group at a time, so the maximum parallelism equals the number of partitions (extra consumers sit idle). If processing one message is slow, it holds up the rest of its partition (**head-of-line blocking**). To scale, add partitions — but that changes the key→partition mapping for new messages. → *More:* Ch 11 — Stream Processing

Q169. What happens when consumers can't keep up with producers? Three options: **drop** messages; **buffer** them in a queue (and decide what happens when the buffer fills — memory, disk, or crash); or apply **backpressure / flow control** (block the producer, like Unix pipes and TCP do). Kafka's log is a large bounded disk buffer: a slow consumer just falls behind (its **consumer lag** grows) without affecting other consumers, until its offset points to data already deleted by retention. So you monitor consumer lag and alert on it. → *More:* Ch 11 — Stream Processing

Q170. ⋆ What is change data capture (CDC) and why use it instead of dual writes? CDC observes all changes written to a database (usually by reading its replication log — PostgreSQL logical decoding, MySQL binlog, MongoDB oplog) and publishes them as a stream that other systems (search index, cache, warehouse) consume in the same order. With **dual writes** — the application writes to the database and then to the search index — you get race conditions (two clients' writes arrive in different orders at the two systems, leaving them permanently inconsistent) and partial failures (one write succeeds, the other fails). CDC makes the database the single leader and everything else a follower. Tools: Debezium, Maxwell, LinkedIn Databus, Kafka Connect. → *More:* Ch 11 — Stream Processing

Q171. How do you bootstrap a new consumer of a CDC stream when the log doesn't go back to the beginning? Take a **consistent snapshot** of the database, remembering its exact position (offset) in the change log, load the snapshot into the new system, then apply the changes from that offset onward. Debezium does this automatically with its initial snapshot. Alternatively, use **log compaction** (Q172), so the log itself always holds the latest value of every key and a new consumer can start from offset 0. → *More:* Ch 11 — Stream Processing

Q172. What is Kafka log compaction? With **log compaction**, Kafka periodically removes older messages that have the same key as a newer one, keeping only the latest value per key; a message with a null value (a **tombstone**) means the key was deleted and is eventually removed too. The log then needs space proportional to the current database size, not the full history, and a new consumer can rebuild a full copy of the data by reading the topic from the start. It is how Kafka Streams stores state changelogs and how CDC topics can serve as a durable source of the latest state. → *More:* Ch 11 — Stream Processing

Q173. ⋆ What is event sourcing? How is it different from CDC? Event sourcing stores all changes to application state as an append-only log of **immutable events** that record *what happened* at the application level ("SeatReserved", "ItemRemovedFromCart"), and derives current state by replaying them. CDC captures low-level row changes from a mutable database that the application uses normally; event sourcing is designed in from the start and the events express user intent. Event sourcing distinguishes **commands** (requests that may be rejected after validation) from **events** (facts that have already happened and cannot be rejected). Snapshots of state avoid replaying the full history every time. → *More:* Ch 11 — Stream Processing

Q174. What is CQRS and how does it relate to event sourcing? CQRS (command query responsibility segregation) separates the model you write to from the model(s) you read from. Writes go to a log (or a normalized write store); one or more **read views** (search index, cache, denormalized tables) are derived from it and optimised for queries. Event sourcing is a natural fit because the event log is the write model and every read view is derived from it — and you can add a new view later by replaying the log. The cost: read views are updated asynchronously, so users may read stale data (handle read-your-writes, see Q71). → *More:* Ch 11 — Stream Processing

Q175. What are the advantages and challenges of keeping data as an immutable event log? Advantages: a full audit trail (accounting ledgers work this way), easier recovery from bugs (you can see and replay what happened), more information than current state alone (a cart item added then removed is useful for analytics), and multiple read views derived from the same log. Challenges: the log grows without bound (compaction and snapshots help), reading your own write in a derived view is asynchronous, and some data must truly be deleted — privacy rules such as GDPR, or illegal content — which needs special mechanisms like Datomic's **excision** or encrypting each user's data and destroying the key (crypto-shredding). → *More:* Ch 11 — Stream Processing

Q176. Event time vs processing time — why does it matter? Event time is when the event actually happened (recorded by the device); **processing time** is when the stream processor handles it. They can differ by seconds to days — queueing, network faults, mobile devices that were offline, a consumer restart, or reprocessing old data. Windowing by processing time produces wrong results (for example, an apparent spike in request rate after a backlog clears). Use event time for correctness, but then you must deal with clocks on devices being wrong: the book suggests logging the device's send time and the server's receive time to estimate and correct the device clock offset. → *More:* Ch 11 — Stream Processing

Q177. ⋆ What are the types of windows in stream processing? Tumbling window: fixed length, non-overlapping (every event in exactly one 1-minute window). **Hopping window**: fixed length, overlapping by a fixed step (5-minute windows every 1 minute). **Sliding window**: all events within some interval of each other (any 5-minute span). **Session window**: no fixed length; groups a user's events that are close together and ends after a period of inactivity (for example 30 minutes) — common for website analytics. → *More:* Ch 11 — Stream Processing

Q178. ⋆ How do you handle late-arriving (straggler) events? What is a watermark? You can never be sure all events for a window have arrived. Options: **ignore** late events (track how many you drop, alert if many), or **publish a correction** — an updated value for the window, retracting the earlier result. Frameworks like Flink and Beam use a **watermark**: a marker in the stream that says "I believe no more events with event time earlier than T will arrive"; when it passes the end of a window, the window is emitted, and an **allowed lateness** setting decides how long late events can still update it. (Watermark

is the frameworks' name; the book describes the same idea as declaring a window ready after a timeout.) → *More:* Ch 11 — Stream Processing

Q179. What are the types of stream joins? Stream–stream (window) join: match events from two streams within a time window — for example, join search events with click events on the same session ID within one hour; the processor keeps state of recent events from both sides. **Stream–table (enrichment) join**: enrich each event with data from a database table, for example add user profile fields to activity events; keep a local copy of the table updated by a CDC stream instead of querying the database remotely each time. **Table–table (materialized view maintenance) join**: both inputs are changelogs and the output is a continuously updated view — like Twitter's timeline cache, joining tweets with follows. → *More:* Ch 11 — Stream Processing

Q180. What is a slowly changing dimension, and why does it make stream joins nondeterministic? In a stream–table join, if the table changes over time (a user changes country, a tax rate changes), the result depends on whether the event is joined with the old or new version — and if events from the two streams interleave differently on a rerun, the output differs. The fix is to give each version of the record a unique ID (version the table), so each event joins with the version that was valid at its event time. Data warehouses call such tables **slowly changing dimensions (SCD)**. The downside is that you can no longer compact old versions away. → *More:* Ch 11 — Stream Processing

Q181. How do stream processors achieve fault tolerance? They cannot just rerun a failed task from the beginning — the stream never ends. Approaches: **microbatching** — split the stream into small batches (about 1 second) and treat each like a mini batch job (Spark Streaming); **checkpointing** — periodically write a consistent snapshot of operator state to durable storage, triggered by **barriers** flowing through the stream, and on failure restart from the last checkpoint (Flink); **rebuilding state** from a compacted changelog topic (Kafka Streams, Samza) or by replaying input. Inside the framework this gives exactly-once *effect*; side effects to external systems need extra care (Q183). → *More:* Ch 11 — Stream Processing

Q182. What are the main uses of stream processing? Complex event processing (CEP): search for patterns of events, like a stored query that runs continuously (fraud patterns). **Stream analytics**: aggregates and statistics over windows — rates, rolling averages, trends — often using probabilistic structures such as HyperLogLog for distinct counts. **Maintaining materialized views**: keeping caches, search indexes and derived tables in sync with a database. **Search on streams**: match each new document against stored queries (alerts for new listings matching a saved search). Plus event-driven workflows between services. → *More:* Ch 11 — Stream Processing

Q183. ⋆ At-most-once, at-least-once, exactly-once — what do they mean? At-most-once: each message is processed zero or one times — commit the offset first, then process; a crash loses messages. **At-least-once**: process first, then commit the offset; a crash after processing but before committing means the message is processed again — duplicates. **Exactly-once** (more precisely **effectively-once**): the final *effect* is as if each message was processed once — messages may physically be processed more than once, but duplicates leave no trace. You get it by making the output and the offset commit atomic, or by making processing **idempotent**. → *More:* Ch 11 — Stream Processing

Q184. ⋆ How does Kafka provide exactly-once semantics? Three pieces. **Idempotent producer** (`enable.idempotence=true`, the default since Kafka 3.0 together with `acks=all`, unless conflicting settings such as `acks=1` are configured): the broker uses a producer ID and per-partition sequence numbers to drop duplicate retries. **Transactions** (`transactional.id`): a producer atomically writes to several partitions *and* commits the consumer offsets in the same transaction — the read-process-write cycle becomes atomic. **Consumers with** `isolation.level=read_committed` see only committed transactional messages. Kafka Streams wraps this as `processing.guarantee=exactly_once_v2`. Limits: it covers Kafka-to-Kafka; writing to an external database still needs idempotent writes or storing the offset in that database's transaction. These features came after the book, which describes the same idea of keeping the atomic commit internal to the stream system. → *More:* Ch 11 — Stream Processing

Exactly-once, idempotency and data integration

Q185. ⋆ How do you make an operation idempotent? An **idempotent** operation has the same effect whether applied once or many times. Some are naturally idempotent (set a key to a value, delete by ID). For others (charge a card, increment a counter), have the client generate a unique **request ID / idempotency**

key (a UUID) once per logical operation and send it with every retry; the server stores processed IDs — ideally in the same transaction as the effect, with a unique constraint — and returns the stored result for duplicates. Stripe-style APIs work this way. For stream consumers, store the last processed offset or event ID together with the output.

```
INSERT INTO payments (request_id, account, amount) VALUES ($1, $2, $3)
ON CONFLICT (request_id) DO NOTHING;   -- retry = no second payment
```

→ *More:* Ch 12 — The Future of Data Systems

Q186. ⋆ What is the transactional outbox pattern? It solves the dual-write problem when a service must update its database *and* publish an event. In the **same local transaction** as the business change, insert the event into an `outbox` table. A separate relay process (polling, or CDC via Debezium reading the outbox table) publishes the outbox rows to the broker and marks them sent. Either both the change and the event are committed, or neither; publishing is at-least-once, so consumers must be idempotent (dedupe by event ID). The book does not use the name "outbox" but describes the same principle: write once to a log-like source of truth, derive everything else from it. → *More:* Ch 12 — The Future of Data Systems

Q187. ⋆ What is a saga, and when do you use it instead of 2PC? A **saga** splits a business transaction across services into a sequence of local transactions, each with a **compensating action** that semantically undoes it (refund the payment, release the seat) if a later step fails. It can be **orchestrated** (a central coordinator tells each service what to do) or **choreographed** (each service reacts to events). Use it across microservices where 2PC is unavailable or would block; the cost is no isolation — other transactions can see intermediate states — so design for it (pending states, idempotent steps, retries). The book's equivalent idea is multi-stage processing of a request through a log (Q189). → *More:* Ch 0a — System Design Fundamentals

Q188. Lambda vs Kappa architecture? Lambda architecture runs two pipelines on the same immutable input: a batch layer (Hadoop/Spark) that produces correct results slowly, and a speed layer (stream processor) that produces approximate results quickly; queries merge both. Problems: you maintain the same logic twice in two frameworks, and merging outputs is hard. **Kappa architecture** uses only a stream processor and a replayable log: to reprocess, start a new job reading the log from the beginning, then switch over. The book favours unifying batch and stream in one system that can do both. → *More:* Ch 12 — The Future of Data Systems

Q189. ⋆ How can you enforce a constraint like "a username must be unique" in a log-based (event-driven) system? Partition the log by the value that must be unique (the username). A single-threaded processor for each partition reads requests in log order; the first request for a username wins and the processor emits "success" or "rejected" events. Because there is a total order per partition, no consensus beyond the log itself is needed. Similarly, a transfer between accounts can be one request event, processed in stages (debit A's partition, credit B's partition), each idempotent by request ID — giving correctness without an atomic multi-partition commit. → *More:* Ch 12 — The Future of Data Systems

Q190. What is the end-to-end argument for databases? Even if each component (TCP, the database's transactions, the stream processor) is exactly-once internally, the system as a whole can still produce duplicates — for example, a user's browser resubmits a form after a timeout, and the server cannot know it is the same request. Duplicate suppression must therefore be done **end to end**: a unique ID generated by the client, carried through every layer and checked at the final destination. Low-level guarantees help performance but cannot replace application-level checks. → *More:* Ch 12 — The Future of Data Systems

Q191. What is the difference between timeliness and integrity? Timeliness means users see the system in an up-to-date state; violations are temporary (stale reads that eventually catch up). **Integrity** means no corruption — no lost or contradictory data, no false information; violations are permanent and need manual repair. The book argues integrity matters much more than timeliness — a card statement missing yesterday's purchase is normal, but a statement whose total does not match its transactions is a serious bug — and that event-based systems can guarantee integrity without coordination, accepting weak timeliness — while traditional ACID transactions tie both together. → *More:* Ch 12 — The Future of Data Systems

Q192. Why do systems often check constraints after the fact rather than before? Many business constraints can tolerate a short violation if you can **apologise and compensate** — airlines overbook flights, warehouses oversell and backorder, banks charge overdraft fees. Checking strictly up front needs coordination (linearizability or consensus), which costs latency and availability. So you may process optimistically, detect violations asynchronously, and run a compensating action — as long as integrity (no lost or duplicated money) is preserved. → *More:* Ch 12 — The Future of Data Systems

Q193. What does "trust, but verify" mean for data systems? Do not assume hardware and software never corrupt data: disks have bit rot, databases have bugs (Jepsen has found consistency bugs in many products), and backups may not be restorable. Build **auditing** in: checksums, periodic comparison of replicas or derived data with the source, end-to-end integrity checks, and actually testing restores. Event-sourced systems make auditing easier because you can replay the log and check that derived state matches; Merkle trees and cryptographic logs help verify large datasets cheaply. → *More:* Ch 12 — The Future of Data Systems

Q194. What is "unbundling the database"? A database bundles features internally: secondary indexes, materialized views, replication logs, caches. "Unbundling" means building the same features across *different* specialised systems (a database, a search index, a cache, a warehouse) connected by a durable, ordered log (Kafka) and stream processors that keep derived views up to date — like a database's internal index maintenance, but at the scale of the whole organisation. The log is the source of truth; derived systems can be rebuilt by replaying it; and **dataflow** replaces dual writes. → *More:* Ch 12 — The Future of Data Systems

Scenario and design questions

Q195. ⋆ Your read replicas are lagging and users see stale data after updating their profile. What do you do? First measure: replication lag metrics and their cause (a long transaction, heavy writes, a slow replica, network). Short-term product fix: **read-your-writes** — route reads of a user's own data to the leader for a short time after they write, or track the write's log position and only read from replicas that have caught up. Longer term: reduce write load on the leader (batching, removing unused indexes), scale replicas, or for critical paths read from the leader. Also check monotonic reads (pin users to one replica). → *More:* Ch 5 — Replication

Q196. ⋆ Design a distributed key-value store (like Dynamo or Cassandra). Clarify: size, read/write ratio, consistency needs, latency. Core design: **partition** with consistent hashing and virtual nodes; **replicate** each key to $n = 3$ nodes on the ring (across racks/zones); **leaderless writes** with tunable quorums (w, r); **conflict handling** with version vectors and siblings, or LWW for append-only data; **sloppy quorum and hinted handoff** for availability; **read repair and anti-entropy** with Merkle trees; **gossip** for membership and failure detection (phi accrual); each node's storage engine is an LSM-tree with WAL, memtable, SSTables, Bloom filters and compaction. Discuss trade-offs: AP vs CP, what happens during a partition, hot keys. → *More:* Ch 5 — Replication

Q197. ⋆ Which database would you choose for X? (how to answer) Do not name a product first. Ask about the **access patterns** (key lookups vs range scans vs ad-hoc analytics vs graph traversals), **read/write ratio**, **consistency and transaction needs** (money, inventory), **data size and growth**, **latency targets**, and the team's operational experience. Then map: relational (PostgreSQL) as a strong default for OLTP with joins and transactions; document stores for self-contained aggregates; wide-column/LSM (Cassandra) for very high write throughput with known queries; a column store or warehouse for analytics; a search engine for text; Redis for caching; a graph DB for deep relationship queries. Mention polyglot persistence kept in sync by CDC. → *More:* Ch 2 — Data Models and Query Languages

Q198. ⋆ Design a system that keeps a search index (or cache) consistent with the primary database. Avoid dual writes from the application. Make the database the source of truth and capture its changes with **CDC** (Debezium → Kafka), partitioned by primary key so updates to a row stay ordered. A consumer applies changes to Elasticsearch/Redis idempotently (upsert by ID, ignore events with an older version or log position). Bootstrap with a snapshot plus the log offset. To rebuild or change the index mapping, replay from a compacted topic into a new index and switch an alias. Monitor consumer lag and run periodic reconciliation jobs that compare counts or checksums. → *More:* Ch 11 — Stream Processing

Q199. ⋆ How would you prevent double-booking of a seat/room/appointment? Options in increasing complexity: a **unique constraint** on (resource, slot) so the second insert fails; `SELECT … FOR UPDATE` on the resource row before checking availability; an **exclusion constraint** for overlapping time ranges in PostgreSQL; **serializable** isolation with retries; for high demand (ticket sales), a short **hold/reservation** with an expiry written atomically, plus a queue in front; in a log-based design, partition requests by resource and process them serially. Name the anomaly — write skew with phantoms — to show you understand why snapshot isolation alone fails. → *More:* Ch 7 — Transactions

Q200. ⋆ A cron job runs on two servers at once after a failover and processes everything twice. How do you fix it? The root cause is two nodes believing they are the leader (a pause or network partition plus timeout-based failover). Fixes: run the scheduler with **leader election** from a coordination service (ZooKeeper, etcd, a Kubernetes Lease); protect the side effects with **fencing tokens** so the stale leader's writes are rejected; and make the job **idempotent** — for example, record each processed item with a unique key in the same transaction as its effect, so a second run does nothing. Never rely on "lock with TTL" alone. → *More:* Ch 8 — The Trouble with Distributed Systems

Q201. ⋆ Design a pipeline that counts page views per minute and is correct under failures and late data. Clients send events with an **event-time** timestamp and a unique **event ID** to an ingestion service that writes to Kafka (partitioned by page ID). A stream processor (Flink / Kafka Streams) does **tumbling 1-minute windows by event time** with a watermark and allowed lateness; late events produce corrected counts (upserts). State is checkpointed; output goes to a store with **idempotent upserts** keyed by (page, minute), so replays do not double count; dedupe on event ID if the client may resend. For exact historical numbers, a nightly batch job recomputes from the raw log and overwrites. Mention approximate distinct counts with HyperLogLog if needed. → *More:* Ch 11 — Stream Processing

Q202. Your database is too slow for writes at peak. Walk through how you'd scale it. Measure first: which queries, locks, I/O or CPU dominate. Then, roughly in order: fix queries and indexes (remove unused indexes — each one slows writes); batch writes and use group commit; move non-critical work to asynchronous queues; scale up the machine (faster disks/SSD); separate reads onto replicas so the primary only takes writes; use a write-optimised (LSM) store for append-heavy data like logs and events; and finally **partition** by a good shard key. State the cost of each step — sharding loses easy joins and cross-shard transactions. → *More:* Ch 6 — Partitioning

Q203. How would you migrate from one database to another with zero downtime?

1. Set up CDC from the old database and do an initial snapshot load into the new one. 2. Keep it in sync from the change stream and verify continuously (row counts, checksums, **shadow reads** comparing results). 3. Switch reads gradually (per feature or percentage) while writes still go to the old system. 4. Switch writes — briefly pause or use a routing flag — and optionally stream changes back to the old database for rollback. 5. Decommission after a safe period. Schema differences are handled in the transformation step; encoding compatibility (Q51) matters throughout. → *More:* Ch 12 — The Future of Data Systems

Q204. ⋆ Explain how you'd make a payments flow safe against retries, timeouts and crashes. Client generates an **idempotency key** per payment attempt; the API stores it with a unique constraint and the request's result. Write the payment intent and an outbox event in one local transaction. Call the payment provider with the same idempotency key (providers support this), and on a timeout **query the status** instead of blindly retrying. Use a state machine (created → authorised → captured → refunded) with only valid transitions, a double-entry ledger, and a reconciliation job that compares your ledger with the provider's reports. This is the end-to-end argument in practice. → *More:* Ch 12 — The Future of Data Systems

Q205. ⋆ What happens to a Raft/ZooKeeper cluster of 5 nodes when the network splits 3–2? The side with 3 nodes is a majority: it keeps (or elects) a leader and continues serving writes and linearizable reads. The 2-node side cannot form a majority, so it cannot elect a leader or commit writes; a leader stranded there stops being able to commit and steps down when it sees a higher term after the partition heals, or sooner if the implementation uses a check-quorum rule as etcd does (its uncommitted entries are discarded). Clients connected to the minority get errors or timeouts — the system chose **C over A** for them. When the network heals, the minority nodes catch up from the leader's log. → *More:* Ch 9 — Consistency and Consensus

Q206. A senior wrap-up: what are the most important trade-offs in data-intensive systems? Reads vs writes (indexes, LSM vs B-tree, normalization vs denormalization); consistency vs latency and availability (linearizability, CAP/PACELC, sync vs async replication); isolation vs throughput (read committed vs snapshot vs serializable); coordination vs scalability (consensus and 2PC vs partitioned logs and idempotency); timeliness vs integrity; and simplicity vs features (one database vs many derived systems). A strong candidate names the trade-off, the workload facts that decide it, and how they would detect and recover when the chosen weakness shows up. → *More:* Ch 12 — The Future of Data Systems

Sources of the question lists

Question wording and frequency were checked against these public lists and discussions (titles only; all answers above are written for this book):

- LeetCode Discuss — "How is *Designing Data Intensive Applications* for Sys Design interview" and system-design / backend interview-experience posts (leetcode.com/discuss)
- donnemartin/system-design-primer — sections on CAP, consistency patterns, availability patterns, replication, sharding (github.com/donnemartin/system-design-primer)
- Public Notion and study-note pages for *Designing Data-Intensive Applications* (chapter-by-chapter notes and question lists)
- DesignGurus — "Explain LSM tree vs B-tree"; PracHub — "B+ Tree vs LSM Tree for Database Interviews"
- TechInterview.org — "What isolation levels really guarantee", MVCC and LSM-tree internals posts
- Educative and CoddyKit — transaction isolation levels; dirty, non-repeatable and phantom reads
- DataVidhya and DataDriven — Kafka / streaming data-engineer interview questions (exactly-once, consumer groups, CDC)
- System Design Handbook — "Distributed systems design interview questions"; DEV Community — consensus algorithms for system design interviews
- Five version facts were checked against official sources in October 2026: ZooKeeper removal in Kafka 4.0 (Kafka 4.0.0 release announcement), the idempotent-producer default since Kafka 3.0 (KIP-679 and the producer config reference), MongoDB's implicit `w: "majority"` default since 5.0 and the map-reduce deprecation in 5.0 (MongoDB manual), and `uuidv7()` in PostgreSQL 18 (PostgreSQL 18 docs). The isolation-anomaly behaviour in Q103, Q108 and Q110 was tested on PostgreSQL 16. Other version and product statements follow the book or general knowledge and were not each re-checked against vendor docs.